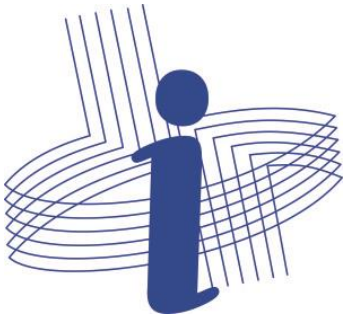




INF 221 – Engenharia de Software I

Aulas 16 e 17. Padrões de Projeto

Cap. 6 - Padrões de Projeto

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>, @engsoftmoderna

Adaptado por Prof. Lucas Veggi e Prof.^a Maria Lúcia Bento Villela



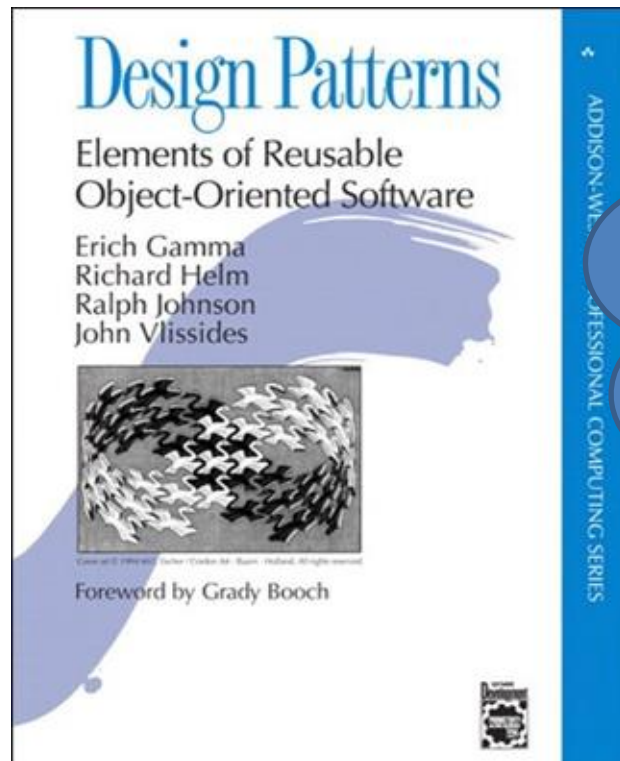
Conteúdo

1. Introdução
2. Fábrica
3. Singleton
4. Proxy
5. Adaptador
6. Fachada
7. Decorador
8. Strategy
9. Observador
10. Template Method
11. Visitor
12. Quando Não Usar Padrões de Projeto



Padrões de Projeto

- Soluções para problemas recorrentes de projeto enfrentados por engenheiros de software



Descrevem objetos e classes que se relacionam para resolver um problema de projeto genérico em um contexto particular.

1994, conhecido como livro da "Gangue dos Quatro" ou GoF

[Shelves](#) > [Computer Science](#) >

Computer Science Books

Showing 1-50 of 17,762

**Introduction to Algorithms (Hardcover)**

by Thomas H. Cormen (shelved 862 times as *computer-science*)
avg rating 4.35 — 9,358 ratings — published 1989

Want to Read ▾

Rate this book

**The Pragmatic Programmer: From Journeyman to Master (Paperback)**

by Andy Hunt (Goodreads Author) (shelved 809 times as *computer-science*)
avg rating 4.33 — 24,305 ratings — published 1999

Want to Read ▾

Rate this book

**Clean Code: A Handbook of Agile Software Craftsmanship (Paperback)**

by Robert C. Martin (shelved 770 times as *computer-science*)
avg rating 4.35 — 23,607 ratings — published 2007

Want to Read ▾

Rate this book

**Structure and Interpretation of Computer Programs (Paperback)**

by Harold Abelson (shelved 693 times as *computer-science*)
avg rating 4.47 — 4,869 ratings — published 1984

Want to Read ▾

Rate this book

**Code: The Hidden Language of Computer Hardware and Software (Paperback)**

by Charles Petzold (shelved 682 times as *computer-science*)
avg rating 4.40 — 10,717 ratings — published 1999

Want to Read ▾

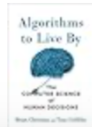
Rate this book

**Design Patterns: Elements of Reusable Object-Oriented Software (Hardcover)**

by Erich Gamma (shelved 556 times as *computer-science*)
avg rating 4.20 — 11,988 ratings — published 1994

Want to Read ▾

Rate this book

**Algorithms to Live By: The Computer Science of Human Decisions (Hardcover)**

by Brian Christian (Goodreads Author) (shelved 525 times as *computer-science*)
avg rating 4.12 — 35,508 ratings — published 2016

Want to Read ▾

Rate this book



11/05/2026



Padrões de Projeto

- **Utilidade #1: Reuso de solução projeto**
 - Suponha que eu tenha um problema de projeto
 - Pode existir um padrão que resolve esse problema
 - **Vantagem:** ganho tempo e não preciso reinventar a roda
 - ✓ Solução já testada e validada
 - ✓ Amplamente utilizada



Padrões de Projeto

- **Utilidade #2: Vocabulário para comunicação**
 - Vocabulário para discussões, documentação etc.

```
public abstract class DocumentBuilderFactory  
extends Object
```

Defines a **factory API** that enables applications to obtain a parser that produces DOM object trees from XML documents.

Uso de padrões de projeto pode ajudar o desenvolvedor a entender o **comportamento** e a **estrutura** da classe que ele precisa usar.



Padrões de Projeto

- **Importante**

- Padrões de projeto visam à criação de projetos de software flexíveis e extensíveis – *design for change*
- Facilitam **mudanças futuras** no código
- Se o “*design for change*” não for uma preocupação, os desenvolvedores correm o **risco** de em breve ter **que planejar um profundo reprojeto de seus sistemas.**



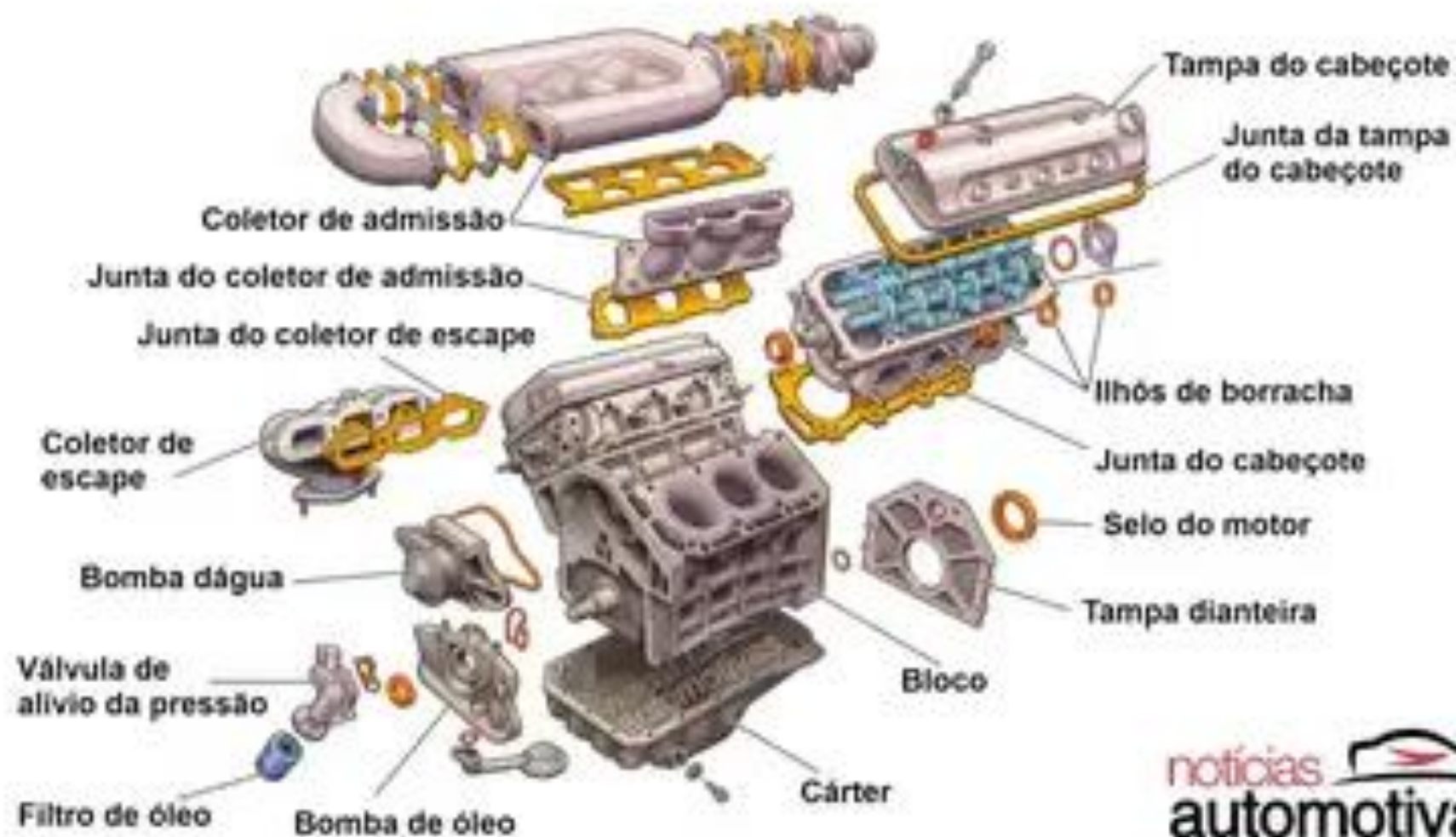
Padrões de Projeto

- **Importante**

- Padrões de projeto visam à criação de projetos de software flexíveis e extensíveis – ***design for change***
- Facilitam **mudanças futuras** no código

E se o código
dificilmente
precisar mudar?

Nesse caso, o uso de
padrões seria um
exemplo de
“overengineering”



Quais são as "peças" que posso usar no design do meu software?

Complementando a comparação
anterior:

Quais são as "peças" que posso usar no
design do meu software para torná-lo
flexível e fácil de modificar?

Livro GoF propõe 23 padrões. Esse material abordará 10 deles

Criacionais	Estruturais	Comportamentais
Abstract Factory Factory Method Singleton Builder Prototype	Proxy Adapter Facade Decorator Bridge Composite Flyweight	Strategy Observer Template Method Visitor Chain of Responsibility Command Interpreter Iterator Mediator Memento State

Criação de objetos

Composição de classes e
objetos

Interação e divisão de responsabilidades
entre classe e objetos



Estrutura e Apresentação dos Padrões de Projeto

- **Contexto** (sistema ou parte de um sistema)
- **Problema** (de projeto)
- **Solução** (por meio de um padrão de projeto)

Padrões são descritos normalmente desta forma.
Acompanhados de códigos exemplificando.

Código no GitHub

<https://github.com/mtov/ESM-ExemplosCodigo/tree/master/cap6>

PADRÕES DE PROJETO

1. FÁBRICA (FACTORY)

Padrão de projeto criacional:
Flexibiliza a criação de objetos



Relembrando a estrutura da explicação

- Contexto
- Problema
- Solução

Contexto: Sistema distribuído que usa canais de comunicação



```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

Em vários pontos do sistema, eu tenho que criar objetos da classe TCPChannel



Problema

- Alguns clientes vão precisar de usar UDP, em vez de TCP
- Como parametrizar as chamadas de **new**?
 - Espalhadas em vários pontos do código, para em alguns casos usar TCPChannel e em outros usar UDPChannel
 - **Problema:** Construtor deve chamar o nome literal da classe do objeto a ser criado.
 - Não permite passar a classe como parâmetro (Java, C++, etc.)
- Como criar um projeto que facilite essa mudança?
 - *Design for change* (projetos flexíveis)
 - Parametrizar a instanciação dos canais de comunicação no código do slide anterior, de forma que ele consiga trabalhar com protocolos diferentes.



Solução: Padrão Fábrica

- Fábrica: método que centraliza a criação de certos objetos

Método **estático** que apenas cria e retorna objetos de uma determinada classe e **oculta** o tipo desses objetos por trás de uma **interface**

```
class ChannelFactory {  
    public static Channel create() { // método fábrica estático  
        return new TCPChannel();  
    }  
}
```

Único ponto de mudança se quisermos usar `UDPChannel` (*Design for change*)

“Aspirador” de métodos `new`: todas as chamadas antigas de `new` migram para uma única chamada, no método fábrica estático.



Solução: Padrão Fábrica

Sem Fábrica

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```



Solução: Padrão Fábrica

Sem Fábrica

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

⇒

Com Método Fábrica Estático

```
void f() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void g() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void h() {  
    Channel c = ChannelFactory.create();  
    ...  
}
```

Métodos clientes usam a **Interface Channel** para manipular objetos criados pelo método Fábrica. TCPChannel e UDPChannel devem implementar essa Interface.

PADRÕES DE PROJETO

2. SINGLETON

Padrão de projeto criacional:
Flexibiliza a criação de objetos



Contexto: Classe Logger

```
void f() {  
    Logger log = new Logger();  
    log.println("Executando f");  
    ...  
}  
void g() {  
    Logger log = new Logger();  
    log.println("Executando g");  
    ...  
}  
void h() {  
    Logger log = new Logger();  
    log.println("Executando h");  
    ...  
}
```



Problema

```
void f() {  
    Logger log = new Logger();  
    log.println("Executando f");  
    ...  
}  
void g() {  
    Logger log = new Logger();  
    log.println("Executando g");  
    ...  
}  
void h() {  
    Logger log = new Logger();  
    log.println("Executando h");  
    ...  
}
```

Problema:

Cada método cliente cria sua própria instância de Logger



Problema

- Como fazer com que todos os clientes usem a mesma instância de `Logger`?
- Na verdade, deveria existir **uma única instância** de `Logger`
 - Que deveria ser **usada em todas as partes do sistema** que precisam registrar algum evento
 - Toda operação (“log”) seria registrada no mesmo arquivo



Solução: Padrão Singleton

- Transformar a classe Logger em Singleton
- **Singleton**: classe que possui no máximo uma instância



Solução: Padrão Singleton

```
class Logger {  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```



Solução: Padrão Singleton

```
class Logger {  
  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```



Solução: Padrão Singleton

```
class Logger {  
  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        1 if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        2 return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```



Solução: Padrão Singleton

```
class Logger {  
  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```



Solução: Padrão Singleton

```
void g() {  
    Logger log = Logger.getInstance();  
    log.println("Executando g");  
    ...  
}  
  
void h() {  
    Logger log = Logger.getInstance();  
    log.println("Executando h");  
    ...  
}
```

Mesma
instância

Por implementação, garante-se
que g() e h() vão trabalhar na
mesma instância da classe

Logger.

PADRÕES DE PROJETO

3. PROXY

Padrão de projeto estrutural:
Flexibiliza a composição de classes e objetos



Contexto: Função que pesquisa livros

```
class BookSearch {  
    ...  
    Book getBook(String ISBN) { ... }  
    ...  
}
```

Principal objetivo da classe BookSearch!

Problema: inserir um cache (para melhorar o desempenho)

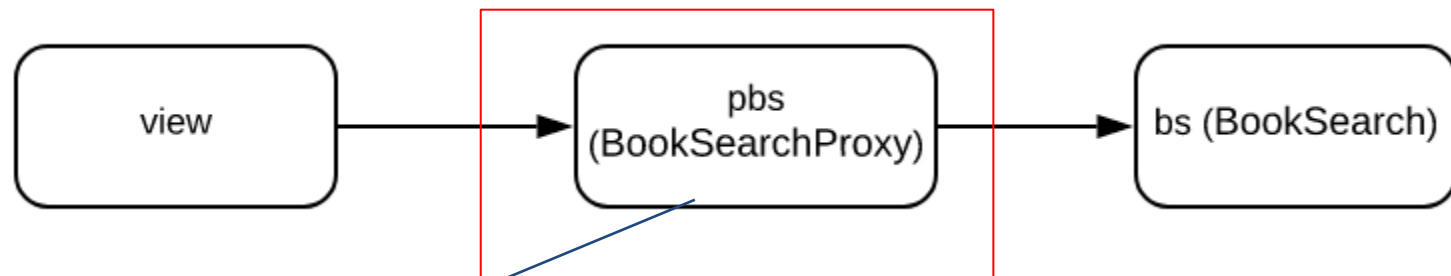


- Se “livro no cache”
 - Retorna livro imediatamente
 - Caso contrário, continua com a pesquisa, segundo a lógica normal do método `getBook()`
- Porém, **não queremos modificar o código de BookSearch**
 - Classe já está funcionando
 - Já tem um desenvolvedor acostumado a mantê-la
 - Queremos mantê-la coesa e aderente ao Princípio de Responsabilidade Única
 - Importante separar, em classes distintas, o interesse “pesquisar livros por ISBN” (requisito funcional) do interesse usar um cache nas pesquisas por livros” (requisito não funcional)



Solução: Padrão Proxy

- **Proxy:** objeto intermediário entre cliente e um objeto base
 - Agrega funcionalidades ao objeto base, sem que ele tome conhecimento
 - Deve implementar as mesmas interfaces do objeto base
- Clientes não “falam” mais com o objeto base (diretamente)
- Eles têm que passar antes pelo proxy

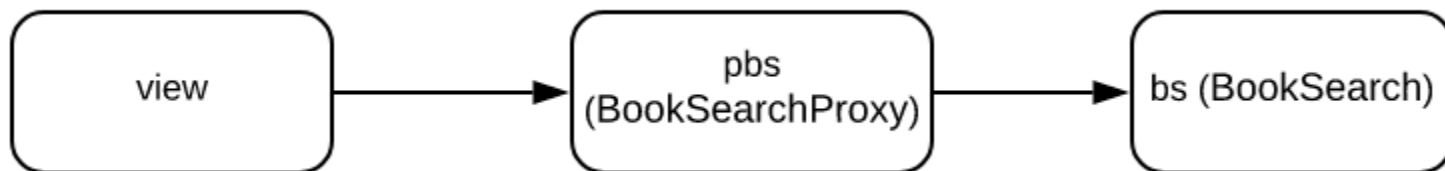


Implementa a lógica do cache



Solução: Padrão Proxy

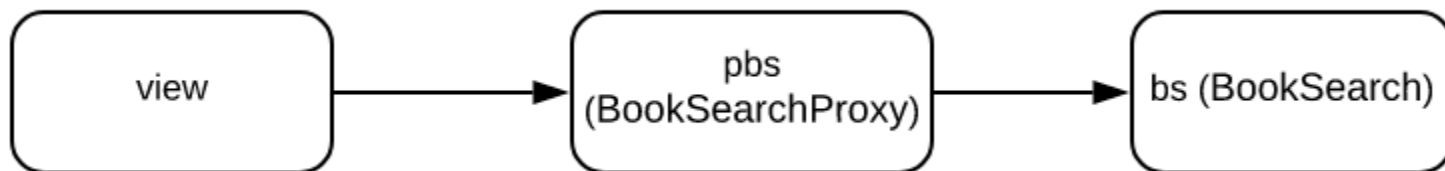
```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs;  
    pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```





Solução: Padrão Proxy

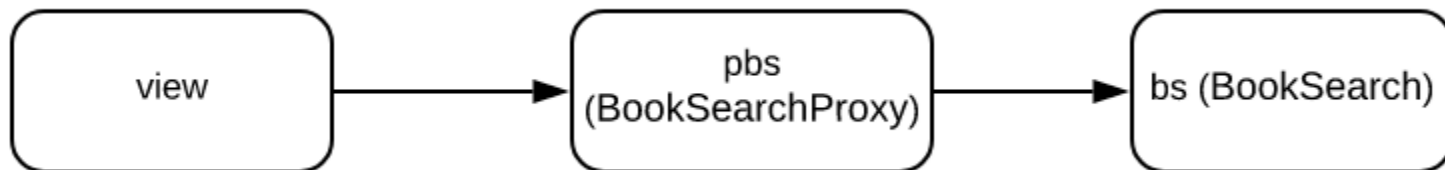
```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs;  
    pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```





Solução: Padrão Proxy

```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs;  
    pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```



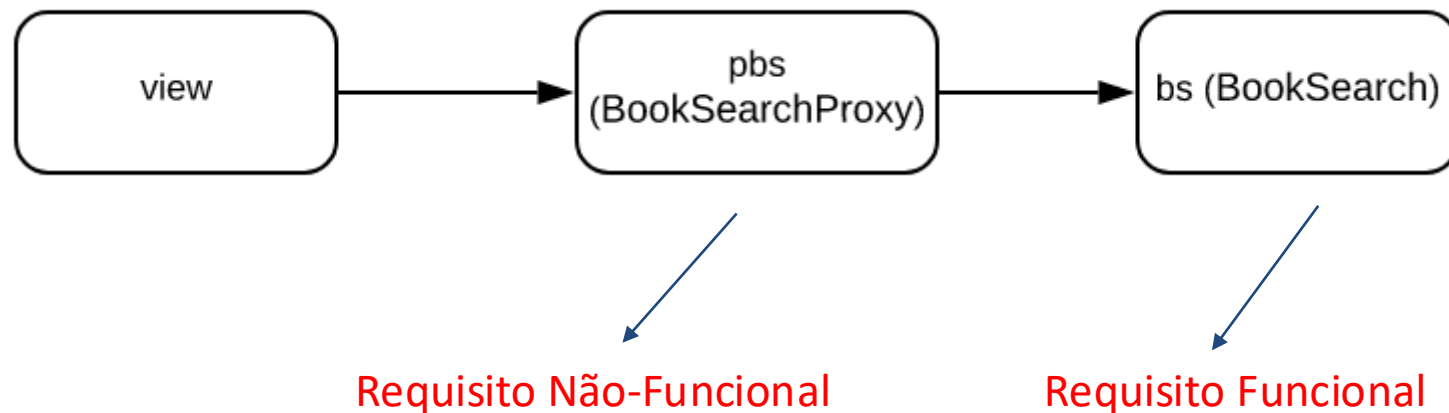
```
class BookSearchProxy implements BookSearchInterface {  
  
    private BookSearchInterface base;  
  
    BookSearchProxy (BookSearchInterface base) {  
        this.base = base;  
    }  
  
    Book getBook(String ISBN) {  
        if("livro com ISBN no cache")  
            return "livro do cache"  
        else {  
            Book book = base.getBook(ISBN);  
            if(book != null)  
                "adicione book no cache"  
            return book;  
        }  
    }  
    ...  
}
```

Objeto base também precisa implementar essa interface

Proxy possui referência para objeto base desde a sua criação

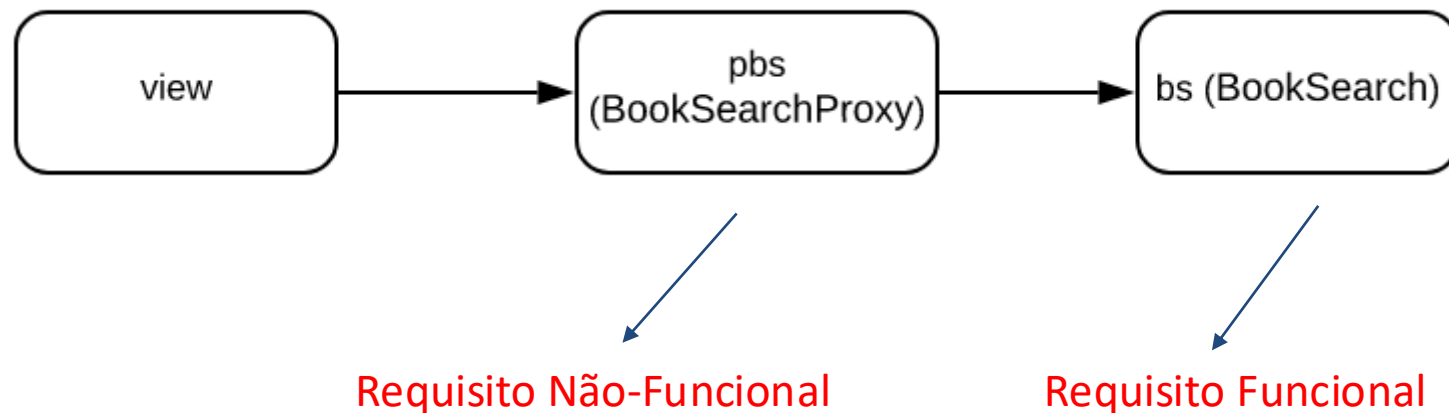
Proxy agrega o requisito não-funcional “Usar cache” ao requisito funcional “Pesquisar livros”

Proxies são ótimos para implementar requisitos não-funcionais!



Logo, um proxy:

- (1) ajuda a melhorar qual propriedade de projeto?
- (2) é um aplicação de qual princípio SOLID?



Logo, um proxy:

- (1) ajuda a melhorar qual propriedade de projeto?
- (2) é um aplicação de qual princípio SOLID?

Integridade Conceitual; Information Hiding; Coesão; **Acoplamento**

Responsabilidade Única, Aberto-Fechado, Substituição de Liskov, Segregação de Interfaces, **Inversão de Dependência (Prefira Interfaces a Classes)**

O código de BookSearchProxy está no livro e também em:

<https://github.com/mtov/ESM-ExemplosCodigo/tree/master/cap6>

PADRÕES DE PROJETO

4. ADAPTADOR (ADAPTER)

Padrão de projeto estrutural:
Flexibiliza a composição de classes e objetos

Contexto: Sistema para Controlar Projetores Multimídia



```
class ProjetorSamsung {  
    public void turnOn() { ... }  
    ...  
}
```

```
class ProjetorLG {  
    public void enable(int timer) { ... }  
    ...  
}
```

Métodos de mesma funcionalidade com assinaturas diferentes!

Poderiam existir mais métodos e mais classes.

Classes fornecidas pelos fabricantes dos projetores



Problema

- No sistema de controle de multimídias, eu gostaria de manipular uma única interface Projetor

```
interface Projetor {  
    void liga() { ... }  
}  
...  
class SistemaControleProjetores {  
    void init(Projetor projetor) {  
        projetor.liga(); // liga qualquer projetor  
    }  
}
```

Clientes sempre usariam objetos dessa interface, sem se preocuparem com a classe concreta que implementa a interface

Sendo mais técnico, eu gostaria de aproveitar e fazer uso de qual princípio SOLID?

Responsabilidade Única, Aberto-Fechado, Substituição de Liskov, Segregação de Interfaces, Inversão de Dependência (Prefira Interfaces a Classes)



Problema

- No sistema de controle de multimídias, eu gostaria de manipular uma única interface Projetor

```
interface Projetor {  
    void liga() { ... }  
}  
...  
class SistemaControleProjetores {  
    void init(Projetor projetor) {  
        projetor.liga(); // liga qualquer projetor  
    }  
}
```

Clientes sempre usariam objetos dessa interface, sem se preocuparem com a classe concreta que implementa a interface

Princípio Prefira Interfaces a Classes



Problema (cont.)

```
class ProjetorSamsung {  
    public void turnOn() { ... }  
    ...  
}  
  
class ProjetorLG {  
    public void enable(int timer) { ... }  
    ...  
}
```

- São classes dos fabricantes dos projetores e **não modificáveis**
 - Eu não tenho acesso a elas, para, por exemplo, fazer com que elas implementem a interface `Projetor`
- Como **compatibilizar** essas interfaces?



Solução: Padrão Adaptador

Recomenda-se usar esse padrão para converter interfaces de classes. **Ex:** Converter Interface `ProjetoR` para interfaces (métodos públicos) das Classes dos Fabricantes.



Solução: Implementar uma Classe Adaptadora



```
class AdaptadorProjektorSamsung implements Projektor {  
  
    private ProjektorSamsung projetor;  
  
    AdaptadorProjektorSamsung (ProjektorSamung projetor) {  
        this.projetor = projetor;  
    }  
  
    public void liga() {  
        projetor.turnOn();  
    }  
  
}
```

Solução: Implementar uma Classe Adaptadora



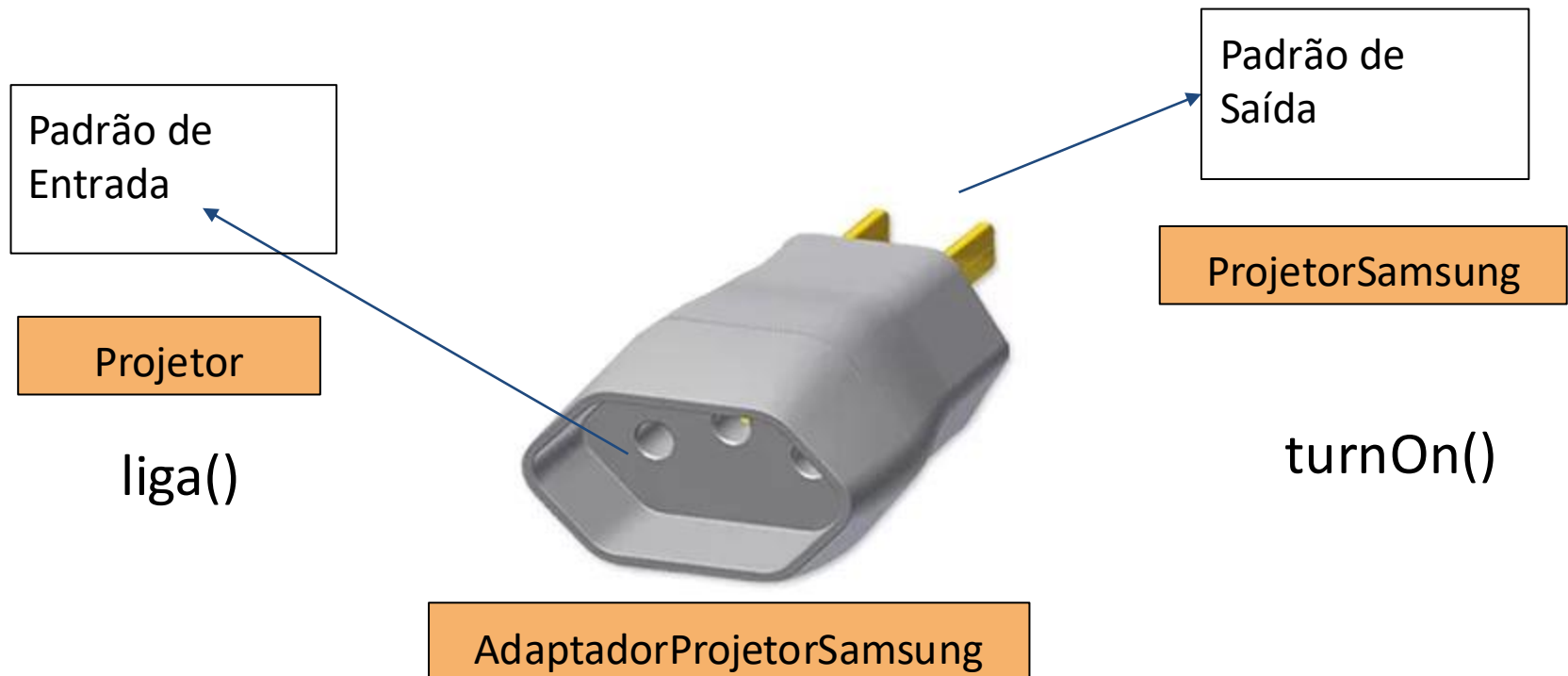
```
class AdaptadorProjektorSamsung implements Projektor {  
    private ProjektorSamsung projetor;  
  
    AdaptadorProjektorSamsung (ProjektorSamung projetor) {  
        this.projetor = projetor;  
    }  
  
    public void liga() {  
        projetor.turnOn();  
    }  
}
```

Objeto real da classe do fabricante
passado como parâmetro no
construtor do Adaptador.

Chama o método do fabricante



Solução: Padrão Adaptador



PADRÕES DE PROJETO

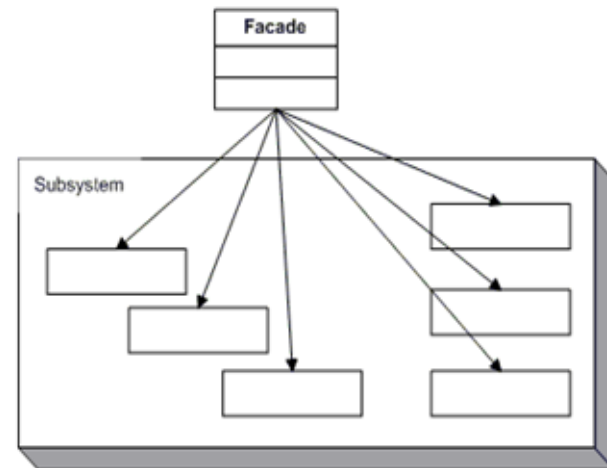
5. FACHADA (FACADE)

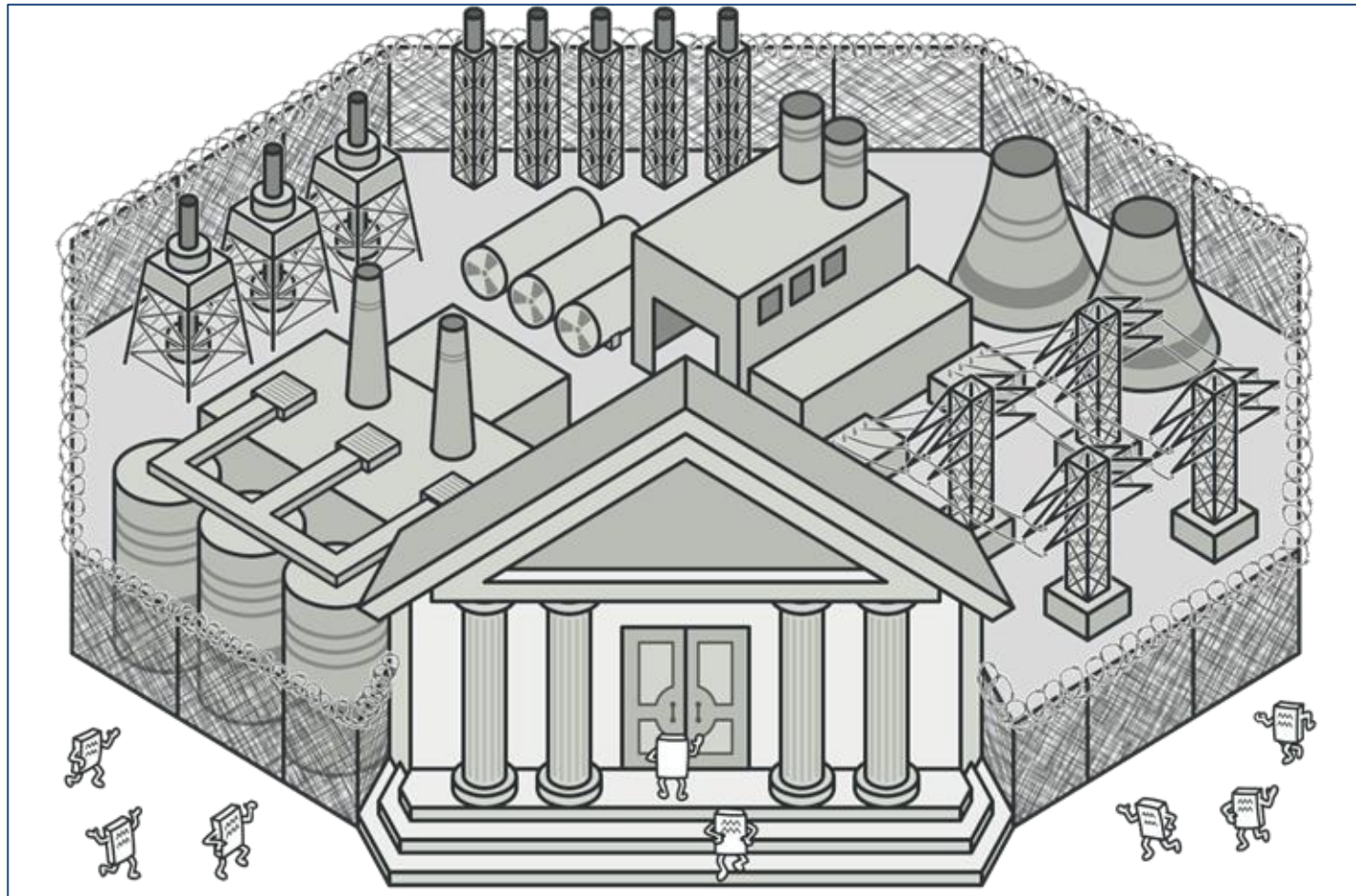
Padrão de projeto estrutural:
Flexibiliza a composição de classes e objetos



Contexto / Problema / Solução

- **Contexto:** módulo M usado por diversos outros módulos
- **Problema:** interface de M é complexa; clientes reclamam que é difícil usar a interface pública de M
- **Solução:** criar uma interface mais simples para M, chamada de **Fachada**.





<https://refactoring.guru/design-patterns/facade>

Exemplo:

**Interpretador de uma linguagem X para
Java**



Contexto

- Suponha que implementamos um interpretador para uma linguagem X a partir de uma linguagem hospedeira, no caso, Java
 - ✓ Imagine que X é uma linguagem de consulta a dados, semelhante a SQL
 - ✓ Para executar programas X, a partir de um código Java, os seguintes passos são necessários:

```
Scanner s = new Scanner("prog1.x");//analizador léxico
Parser p = new Parser(s);
AST ast = p.parse(); //analizador sintático
CodeGenerator code = new CodeGenerator(ast);
code.eval(); //gerador de código
```




Problema

- É necessário **conhecer classes internas do interpretador X** para converter código.
 - **Dificuldade desnecessária**
 - Melhor criar uma **interface mais simples** para chamar o interpretador da linguagem X
 - Criar uma “**Fachada**” para encapsular o interpretador.



Solução: Padrão Fachada

```
Scanner s = new Scanner("prog1.x"); //analizador léxico  
Parser p = new Parser(s);  
AST ast = p.parse(); //analizador sintático  
CodeGenerator code = new CodeGenerator(ast);  
code.eval(); //gerador de código
```



```
new InterpretadorX("prog1.x").eval();
```

Interface mais simples,
que a de cima

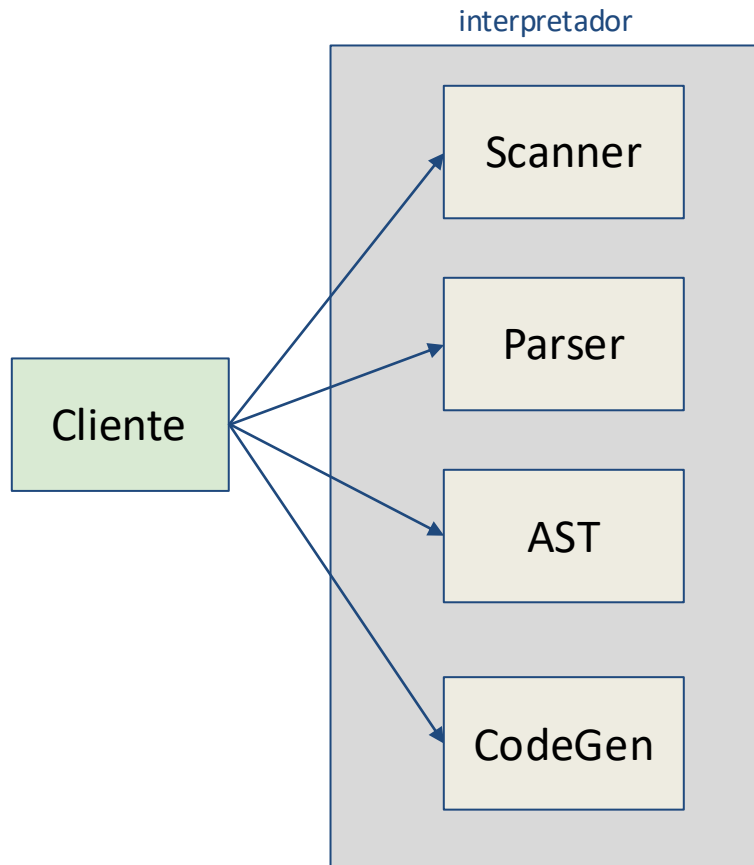


Solução: Padrão Fachada

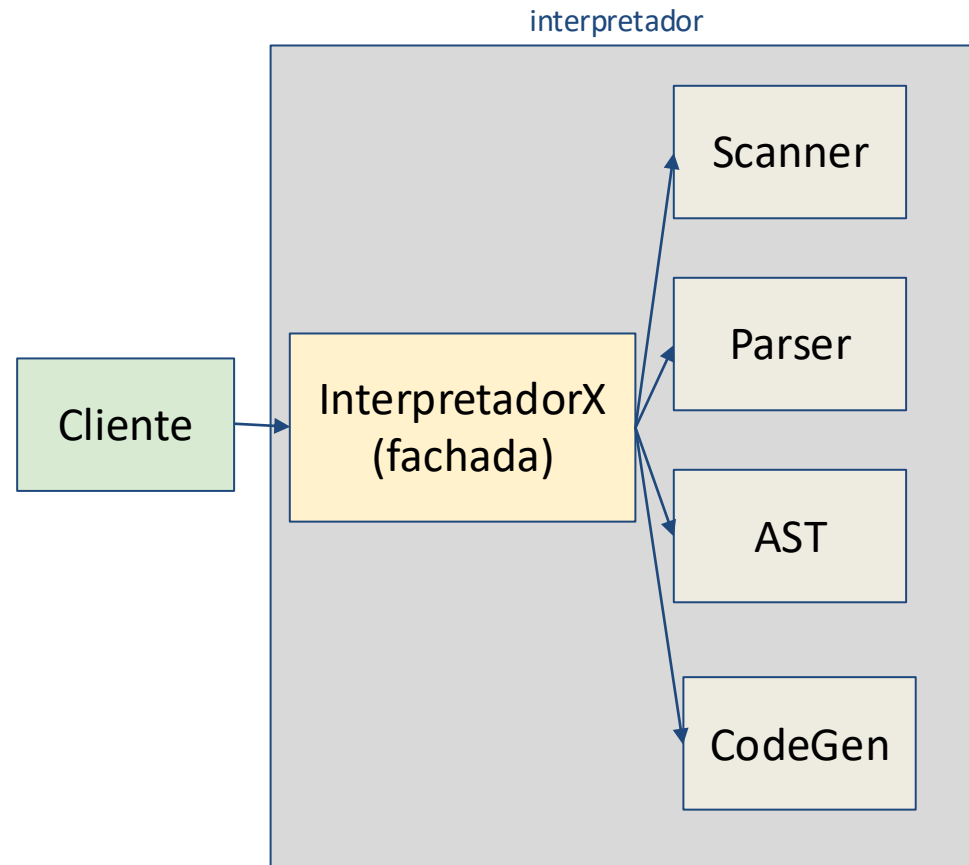
```
class InterpretadorX {  
  
    private String arq;  
  
    InterpretadorX(arq) {  
        this.arq = arq;  
    }  
  
    void eval() {  
        Scanner s = new Scanner(arq);  
        Parser p = new Parser(s);  
        AST ast = p.parse();  
        CodeGenerator code = new CodeGenerator(ast);  
        code.eval();  
    }  
}
```

Basta chamar esse método da Fachada para interpretar um código X!

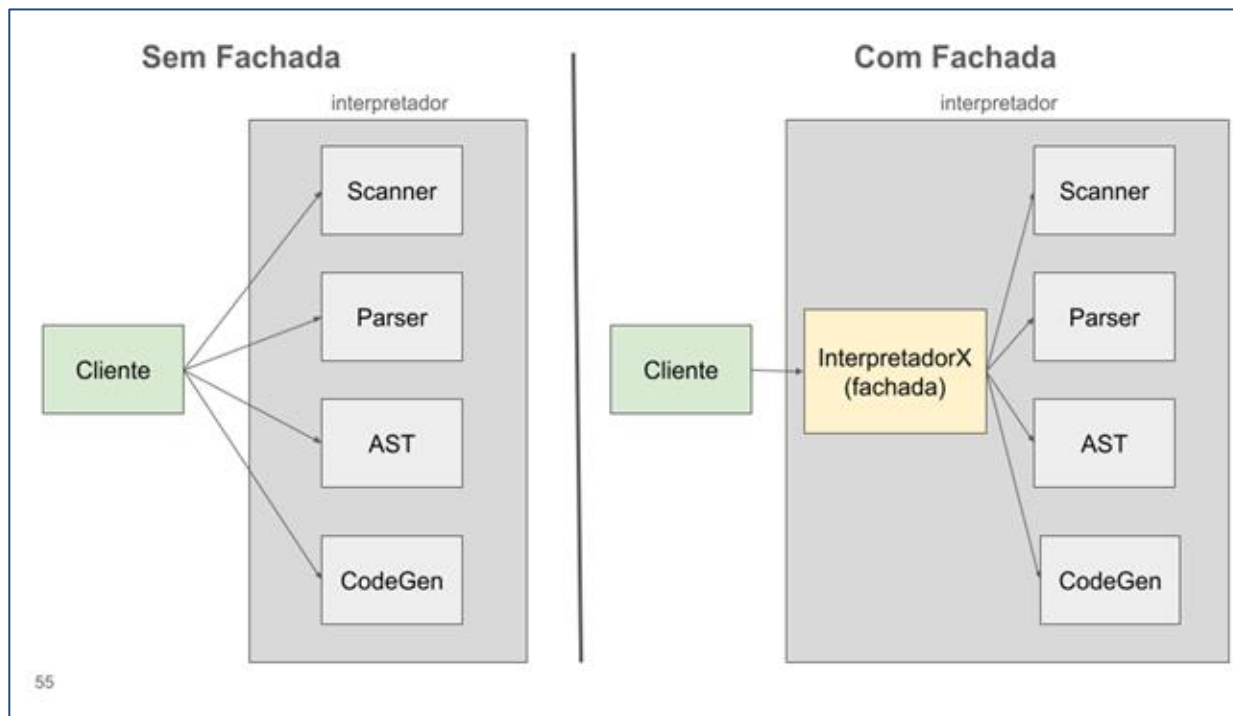
Sem Fachada



Com Fachada

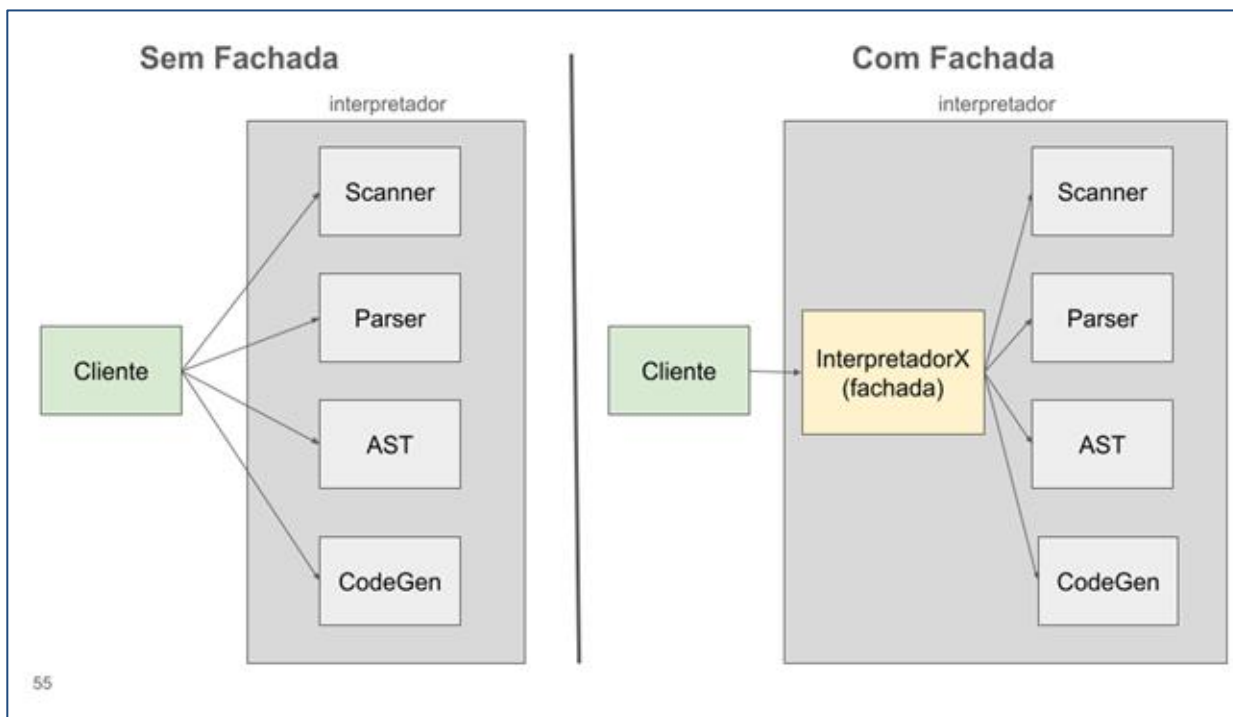


Na figura anterior, considerando o cliente, qual propriedade do seu projeto estamos melhorando?



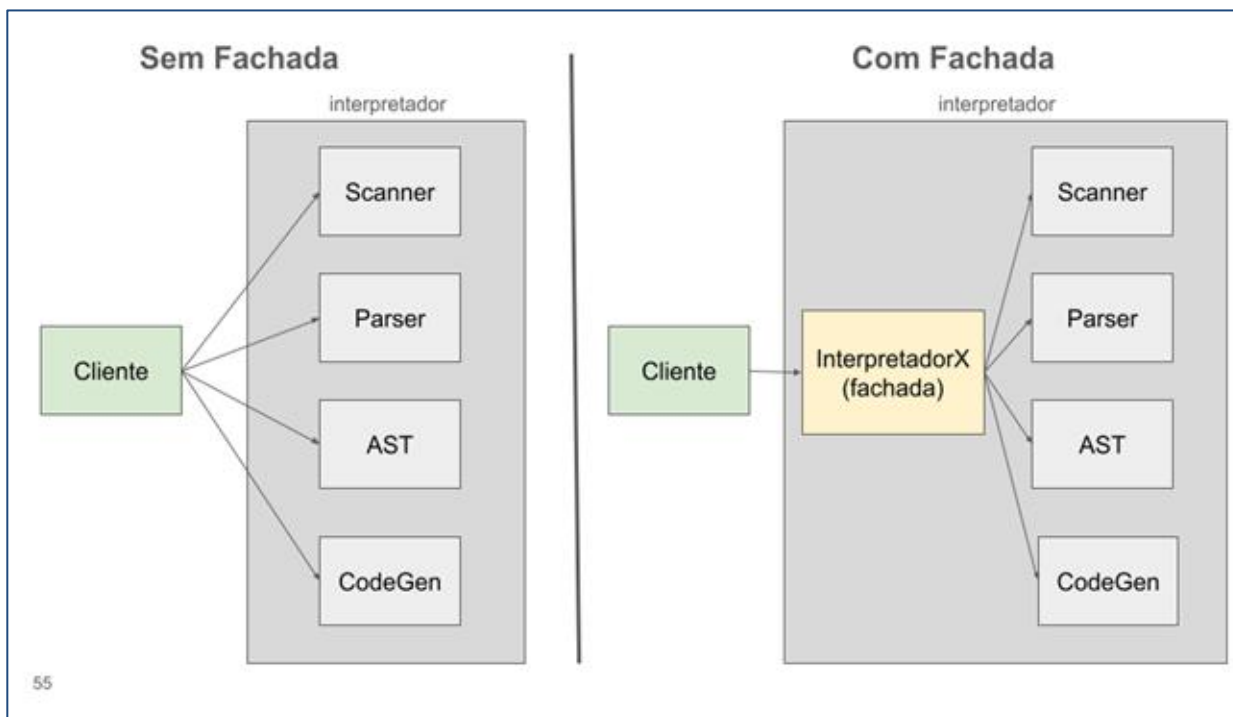
Integridade Conceitual; Ocultamento de Informação; Coesão; Acoplamento

Na figura anterior, considerando o cliente, qual propriedade do seu projeto estamos melhorando?



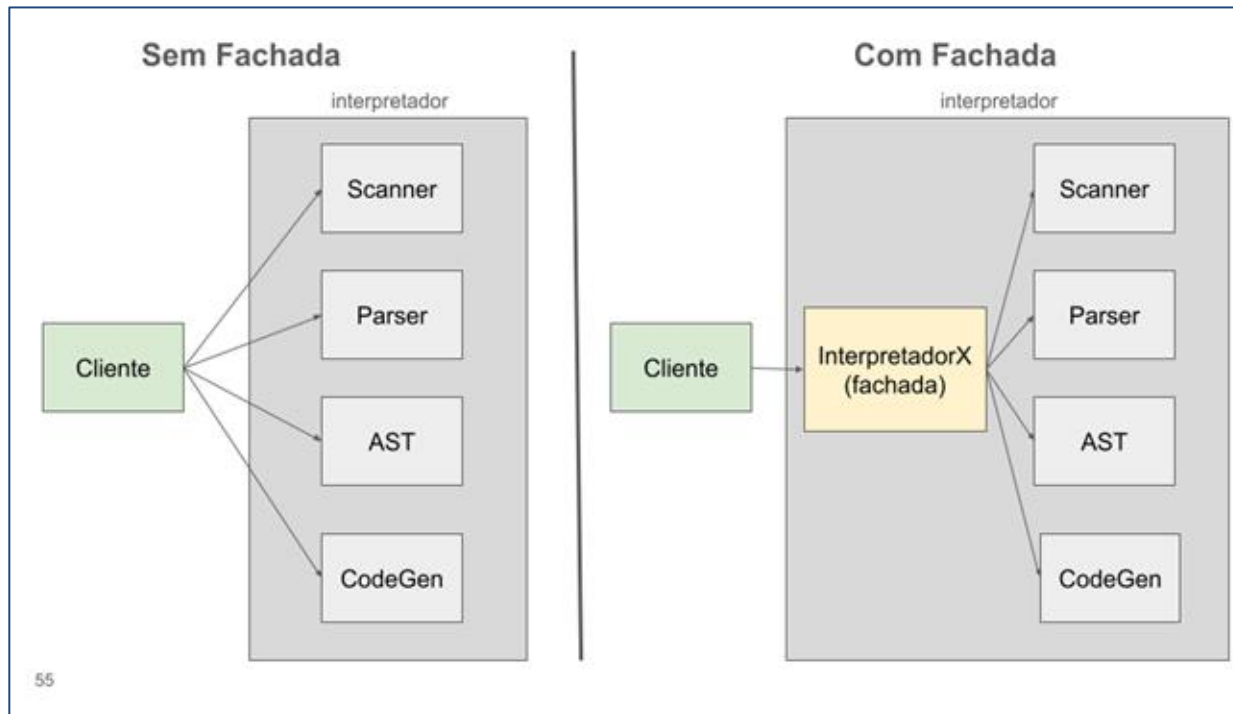
Integridade Conceitual; **Ocultamento de Informação**; Coesão; Acoplamento

Por outro lado, se **não** tomarmos cuidado, qual princípio de projeto pode ser violado quando se usa uma fachada?



Responsabilidade Única, Aberto-Fechado, Substituição de Liskov, Segregação de Interfaces, Inversão de Dependência (Prefira Interfaces a Classes)

Por outro lado, se **não** tomarmos cuidado, qual princípio de projeto pode ser violado quando se usa uma fachada?



Responsabilidade Única, Aberto-Fechado, Substituição de Liskov, Segregação de Interfaces, Inversão de Dependência (Prefira Interfaces a Classes)

PADRÕES DE PROJETO

6. DECORATOR

Padrão de projeto estrutural:
Flexibiliza a composição de classes e objetos

Contexto: Sistema que usa canais de comunicação

- Já foi usado para explicar o padrão Fábrica

```
interface Channel {  
    void send(String msg);  
    String receive();  
}  
  
class TCPChannel implements Channel {  
    ...  
}  
  
class UDPChannel implements Channel {  
    ...  
}
```



Problema: Precisamos adicionar funcionalidades extras em canais

- Em muitos usos, os canais "default" (TCP ou UDP) não são suficientes
- Por exemplo, precisamos também de canais com:
 - Compactação e descompactação de dados
 - Buffers (ou caches) de dados
 - Logging dos dados enviados e recebidos etc.
- Essas funcionalidades são opcionais e combináveis
 - É possível usar algumas, uma ou nenhuma.



Possível Solução: via herança

- Subclasses são criadas para cada conjunto de novas funcionalidades
 - `TCPZipChannel extends TCPChannel`
 - `TCPBufferedChannel extends TCPChannel`
 - `TCPBufferedZipChannel extends TCPZipChannel extends TCPChannel`
 - `TCPLogChannel extends TCPChannel`
 - `TCPLogBufferedZipChannel extends TCPBufferedZipChannel extends TCPZipChannel extends TCPChannel`



Possível Solução: via herança

- E precisamos de classes similares para UDP...
- `UDPZipChannel extends UDPChannel`
- `UDPBufferedChannel extends UDPChannel`
- `UDPBufferedZipChannel extends UDPZipChannel extends UDPChannel`
- `UDPLogChannel extends UDPChannel`
- `UDPLogBufferedZipChannel extends UDPBufferedZipChannel extends`
`UDPZipChannel extends UDPChannel`

Problema: solução inviável devido à explosão combinatória do número de subclasses!!!



Solução: Padrão Decorador

- Resolve o nosso problema -- que é adicionar novas funcionalidades em uma classe -- por meio de **composição** e não mais por meio de herança
 - Sem gerar um número excessivo de classes
- Princípio **Prefira Composição à herança.**



Solução: Padrão Decorador

- Exemplo 1: cliente configurando canal

```
channel = new ZipChannel(new TCPChannel());  
// TCPChannel que compacte/descompacte dados enviados/recebidos
```

Configuração de Channel no momento da sua instanciação. O `new` mais interno sempre cria um objeto da classe a ser decorada. **Ex:** `TCPChannel` ou `UDPChannel`



Solução: Padrão Decorador

- Exemplo 2: cliente configurando canal

```
channel = new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```

Configuração de Channel no momento da sua instanciação. O new mais interno sempre cria um objeto da classe a ser decorada. **Ex:** TCPChannel ou UDPChannel



Solução: Padrão Decorador

- Comparação

```
channel = new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```



Dentro de uma caixa, tem outra caixa, que tem outra caixa ... até chegar no presente. Isto é, até chegar em TCPChannel ou UDPChannel



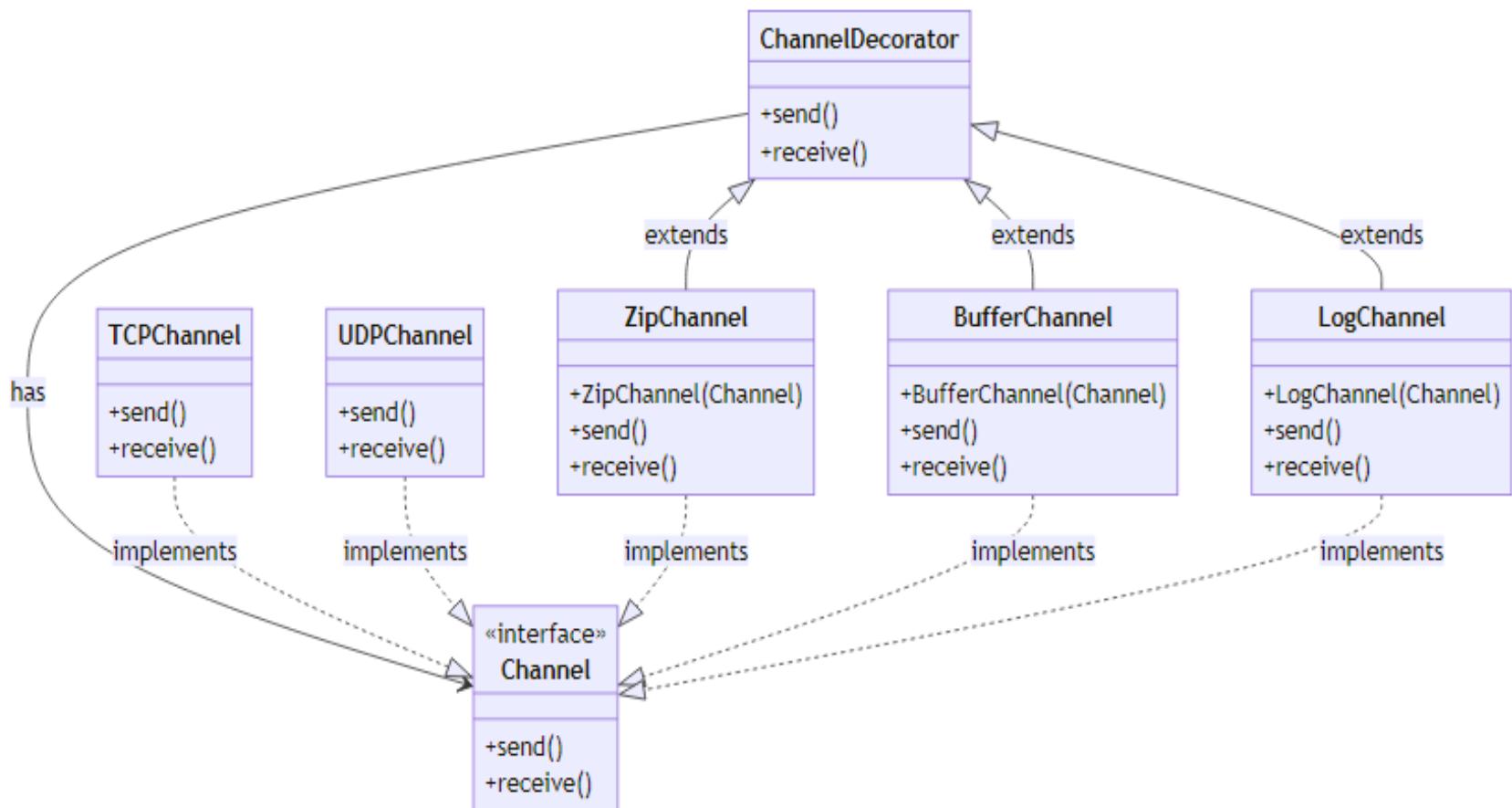
Implementação do Decorator

- Uma classe base, chamada `ChannelDecorator`
- Uma subclasse para cada funcionalidade decorativa
 - Exemplos: `ZipChannel`, `BufferedChannel`, `LogChannel`



Implementação do Decorator

- Não são mais necessárias:
 - Subclasses para combinações de funcionalidades.
 - Exemplo: `TCPBufferedZipChannel`
- Logo, não existe explosão no número de subclasses





ChannelDecorator

```
class ChannelDecorator implements Channel {  
    private Channel channel;  
    public ChannelDecorator(Channel channel) {  
        this.channel = channel;  
    }  
    public void send(String m) {  
        channel.send(m);  
    }  
    public String receive() {  
        return channel.receive();  
    }  
}
```

Todos os decoradores reais
são subclasses dessa classe
base

Possui internamente um Channel
para o qual delega as chamadas de
send e receive. Métodos da
interface Channel

Sempre que for esperado um objeto
Channel, podemos passar um
ChannelDecorator no lugar.

Implementação de decorador real: ZipChannel



ZipChannel

```
class ZipChannel extends ChannelDecorator {  
  
    public ZipChannel(Channel c) {  
        super(c);  
    }  
  
    public void send(String msg) {  
        "compacta mensagem msg"  
        super.channel.send(msg);  
    }  
  
    public String receive() {  
        String msg = super.channel.receive();  
        "descompacta mensagem msg"  
        return msg;  
    }  
}
```

Construtor da classe pai
(ChannelDecorator).



ZipChannel

```
class ZipChannel extends ChannelDecorator {  
  
    public ZipChannel(Channel c) {  
        super(c);  
    }  
  
    public void send(String msg) {  
        "compacta mensagem msg"  
        super.channel.send(msg);  
    }  
  
    public String receive() {  
        String msg = super.channel.receive();  
        "descompacta mensagem msg"  
        return msg;  
    }  
  
}
```

Redefine o método `send`, para, antes de enviar a `msg`, compacta-la.



ZipChannel

```
class ZipChannel extends ChannelDecorator {  
  
    public ZipChannel(Channel c) {  
        super(c);  
    }  
  
    public void send(String msg) {  
        "compacta mensagem msg"  
        super.channel.send(msg);  
    }  
  
    public String receive() {  
        String msg = super.channel.receive();  
        "descompacta mensagem msg"  
        return msg;  
    }  
}
```

Redefine o método receive, para descompactar a msg, depois que ela é recebida.



ZipChannel

```
class ZipChannel extends ChannelDecorator {  
  
    public ZipChannel(Channel c) {  
        super(c);  
    }  
  
    public void send(String msg) {  
        "compacta mensagem msg"  
        super.channel.send(msg);  
    }  
  
    public String receive() {  
        String msg = super.channel.receive();  
        "descompacta mensagem msg"  
        return msg;  
    }  
  
}
```

Decoradores reais fazem o papel que lhes cabem nos métodos sobrescritos e também chamam as versões originais dos métodos herdados da superclasse.

A superclasse, por sua vez, delega as atividades para o seu `Channel` interno.



Exemplo 1

- Para entender o funcionamento de ZipChannel, suponha o seguinte código:

```
Channel c = new ZipChannel (new TCPChannel());  
c.send("Hello, world");
```

Chamada de `send` dispara as seguintes execuções de métodos:

- Primeiro, executa-se `ZipChannel.send`, que vai compactar a imagem
- Após a compactação, `ZipChannel.send` chama `super.channel.send`, que vai executar `ChannelDecorator.send` (`ChannelDecorator` é superclasse de `ZipChannel`)
- `ChannelDecorator.send` apenas repassa a chamada para o `Channel` por ele referenciado, que no caso é um `TCPChannel`
- Finalmente, chegamos ao `TCPChannel.send`, que vai transmitir a mensagem via TCP.



Exemplo 2

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```

```
class BufferChannel extends ChannelDecorator {  
    ... super.channel  
}
```



Exemplo 2

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```

```
class ZipChannel extends ChannelDecorator {  
    ... super.channel  
}
```

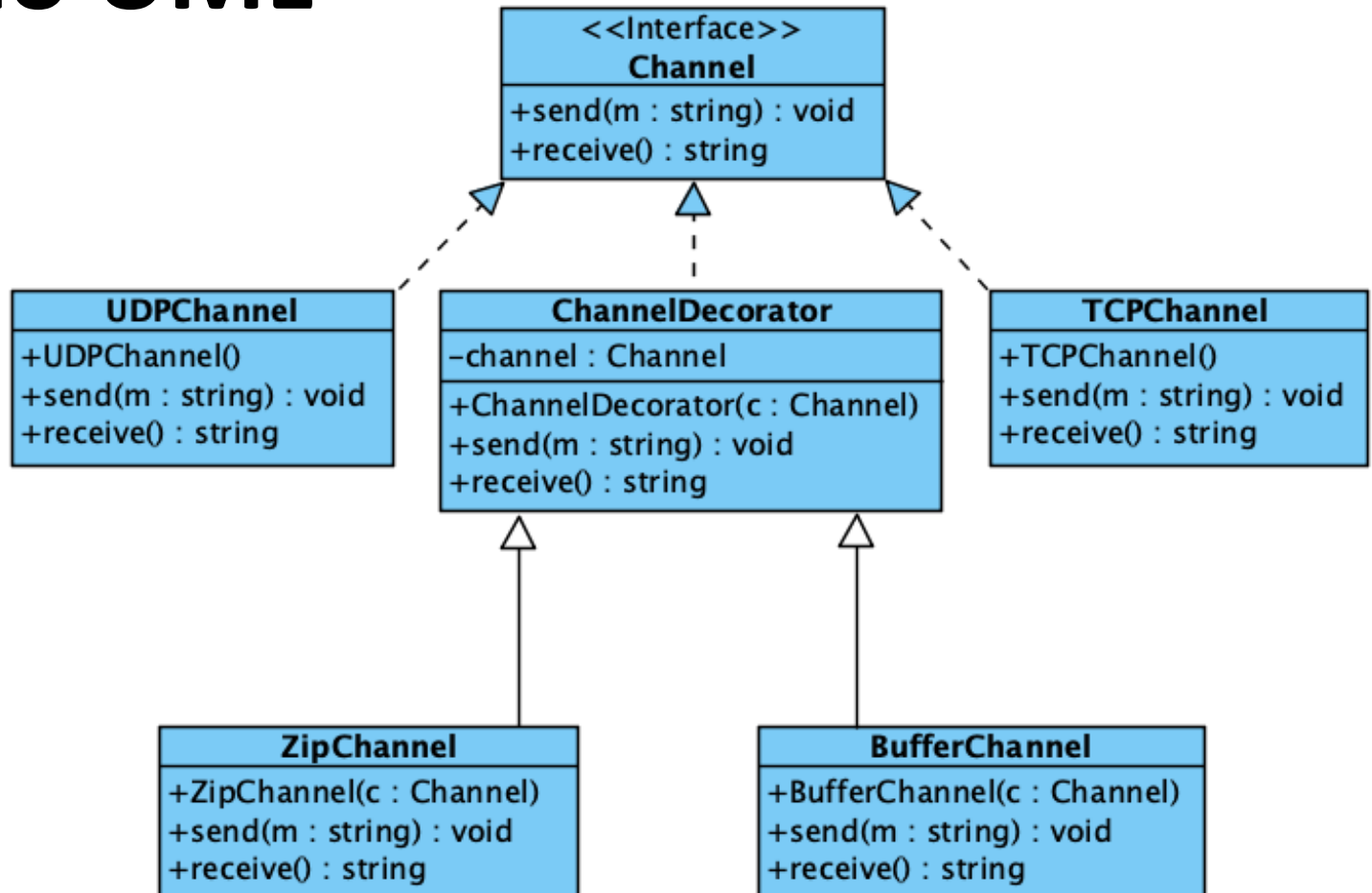


Exemplo 2

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```

```
class TCPChannel implements Channel {  
    // canal "final"  
}
```

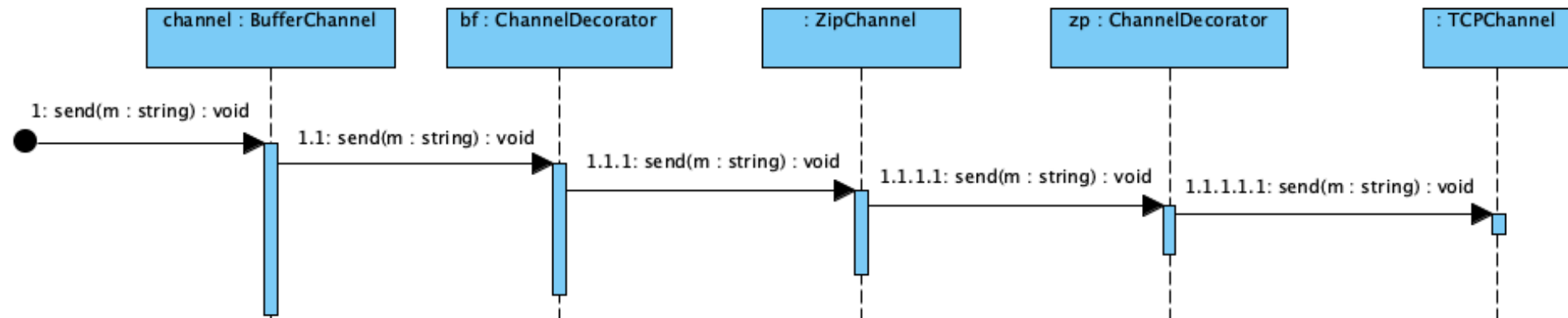
Exemplo UML



Exemplo UML

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```

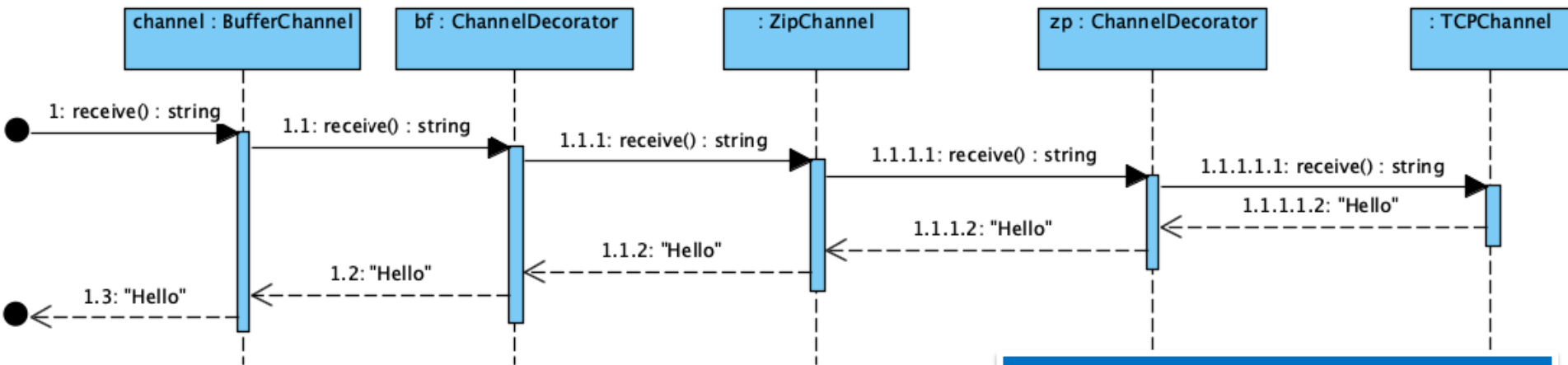
```
channel.send("Testando o canal...");
```



Exemplo UML

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```

```
String msg = channel.receive();
```



Demonstrar no Eclipse IDE
[PadraoDeProjetoDecorator]

Código Decorator:

<https://gist.github.com/mtov/c8d65378a2904af01c20c53922f5ae1d>

PADRÕES DE PROJETO

7. STRATEGY

Padrão de projeto comportamental:
Flexibiliza interação e divisão de
responsabilidades entre classe e objetos

Contexto: Biblioteca de Estruturas de Dados

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    public void sort() {  
        ... // ordena a lista usando Quicksort  
    }  
  
}
```



Problema

- Clientes da classe estão precisando usar novos algoritmos de ordenação
 - Ex.: ShellSort, HeapSort, InsertionSort etc.
- Classe `MyList` **não** está aberta a extensões
- `MyList` **não segue** o princípio Aberto/Fechado, considerando o algoritmo de ordenação.



Solução: Padrão Strategy

- **Objetivo: parametrizar os algoritmos** usados por uma classe
 - Tornar uma classe “**aberta**” a **novos algoritmos** (sem alterar o seu código)
 - No exemplo, a novos algoritmos de ordenação
- Prescreve como **encapsular** uma família de algoritmos e como torná-los intercambiáveis
- Seu uso é recomendado quando uma classe é usuária de um certo algoritmo.
 - Porém, como existem diversos algoritmos com o mesmo propósito, não se quer antecipar uma decisão e implementar apenas um deles no corpo da classe (como ocorre na primeira versão de `MyList`).

Passo #1: Criar uma hierarquia de "estratégias"
(estratégia = algoritmo)



Solução: Padrão Strategy

Também poderia ser uma interface

Objeto do Cliente da Estratégia

```
abstract class SortStrategy {  
    abstract void sort(MyList list);  
}  
  
class QuickSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}  
  
class ShellSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}
```

Código do QuickSort

Código do ShellSort

Passo #2: Modificar MyList para usar a hierarquia de estratégias



Solução: Padrão Strategy

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search
```

```
    private SortStrategy strategy;
```

Algoritmo de ordenação virou
um atributo da classe!

```
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }
```

QuickSort é o *default*

```
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }
```

Altera o algoritmo

```
    public void sort() {  
        strategy.sort(this);  
    }
```

Ordena com a estratégia
escolhida

PADRÕES DE PROJETO

8. OBSERVADOR (OBSERVER)

Padrão de projeto comportamental:
Flexibiliza interação e divisão de
responsabilidades entre classe e objetos

Contexto: Sistema de uma Estação Meteorológica



- Duas **classes** desse sistema:
 - **Temperatura** (*model*) - armazena as temperaturas monitoradas da estação meteorológica
 - **Termômetro** (*view*) – cria objetos visuais que exibem as temperaturas sob monitoramento
- Existem diversos tipos de Termômetro (digital, analógico, web, celular, console etc.)
- Termômetros devem exibir a temperatura atual
 - Se a temperatura mudar, todos os termômetros devem ser atualizados imediatamente



Problema

- **Não queremos acoplar** Temperatura (*model*) a Termômetros (*view*)
 - Temperatura não vai saber quais termômetros estão tendo acesso a ela
- **Motivos:**
 - Interfaces gráficas (*view*) mudam com frequência, portanto são instáveis (acoplamento ruim).
 - Buscamos tornar classe de dados (*model*) independente de classes de apresentação (*view* ou interface com usuários)
 - E tornar flexível a adição de um novo tipo de termômetro no sistema, sem precisar mexer com o código de Temperatura



Solução: Padrão Observador

- Implementa uma relação do tipo *um-para-muitos* entre objetos chamados de **Sujeito** e **Observadores**
 - **Sujeito**: Temperatura (*model*)
 - **Observadores**: Termômetros (*view*)
- Quando o **estado** de um Sujeito **muda**, seus Observadores são **notificados** imediatamente.
- Mas **Sujeito não conhece o tipo concreto de seus Observadores**



Solução: Padrão Observador

- Programa Principal

```
void main() {  
    Temperatura t = new Temperatura();  
    t.addObserver(new TermometroCelsius());  
    t.addObserver(new TermometroFahrenheit());  
    t.setTemp(100.0);  
}
```

Dois observadores

Sujeito

Modifica o estado do sujeito - notifica os
termômetros (observadores).

Por *default*, temperatura é monitorada em °C. Cada observador pode fazer as conversões necessárias. 105



Solução: Padrão Observador

- Interface Observer

Esta interface deve ser
implementada por todas
classes Observadoras

```
public interface Observer {  
    public void update(Subject s);  
}
```

Sempre que um Sujeito é alterado, este método é chamado em todos os seus Observadores para atualizar as Views.



Solução: Padrão Observador

- **Classe Subject**

Todos os **Sujeitos** devem herdar
desta classe abstrata

```
public abstract class Subject {  
    private List<Observer> observers = new ArrayList<Observer>();  
    public void addObserver(Observer o) {  
        this.observers.add(o);  
    }  
    public void removeObserver(Observer o) {  
        this.observers.remove(o);  
    }  
    protected void notifyObservers() {  
        for(Observer o : this.observers) {  
            o.update(this);  
        }  
    }  
}
```

Sempre que um Sujeito é alterado, este método deve ser chamado por ele para avisar os seus Observadores



Solução: Padrão Observador

- Classe Temperatura

```
class Temperatura extends Subject {  
    private double temp;  
  
    public double getTemp() {  
        return temp;  
    }  
  
    public void setTemp(double temp) {  
        this.temp = temp;  
        notifyObservers();  
    }  
}
```

Classe que implementa
addObservers e
notifyObservers

Notifica todos os
observadores (isto é,
Termômetros) que
foram adicionados a
uma Temperatura



Solução: Padrão Observador

- Uma classe Termômetro

```
class TermometroCelsius implements Observer{  
  
    public void update(Subject s){  
        double temp = ((Temperatura) s).getTemp();  
        System.out.println("Temperatura Celsius: " + temp);  
    }  
  
}
```

Todos Observadores devem implementar esse método. Chamada de **notifyObservers** (em Temperatura) resulta na execução de **update** de cada um de seus observadores

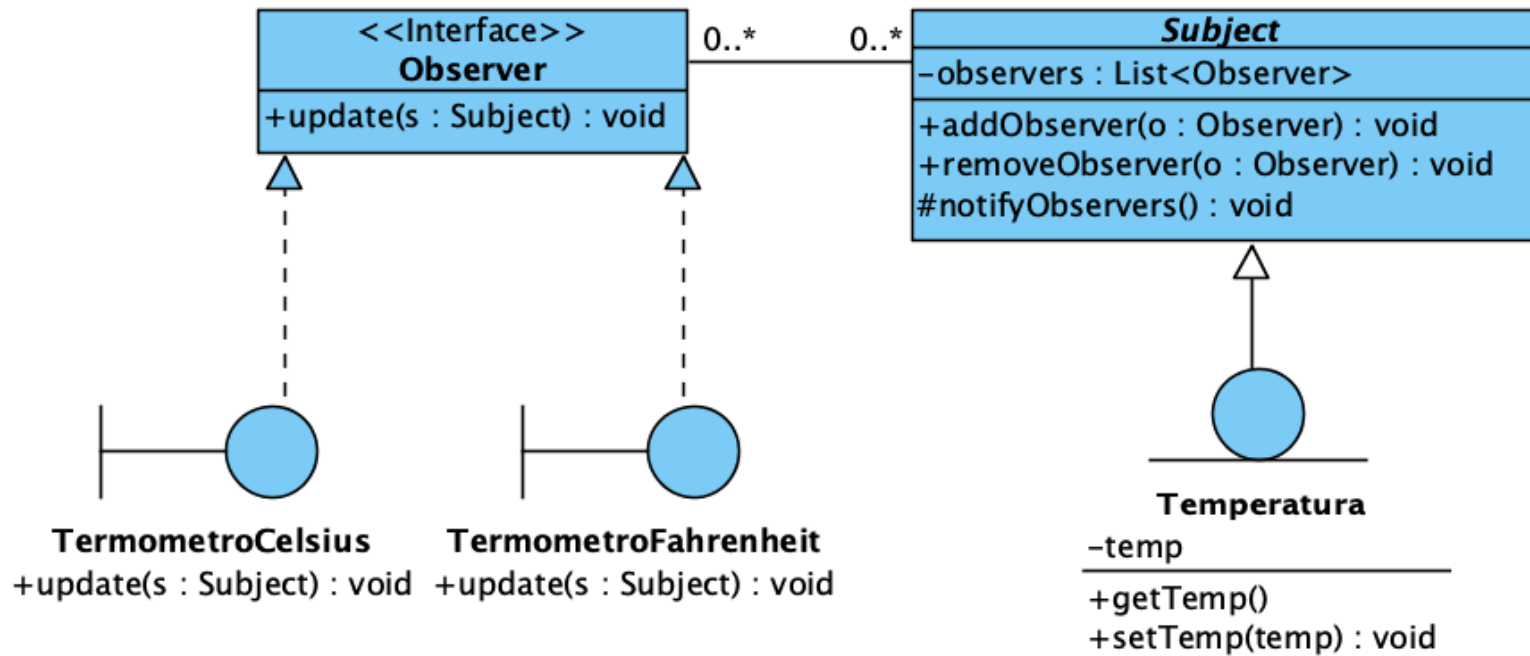


Solução: Padrão Observador

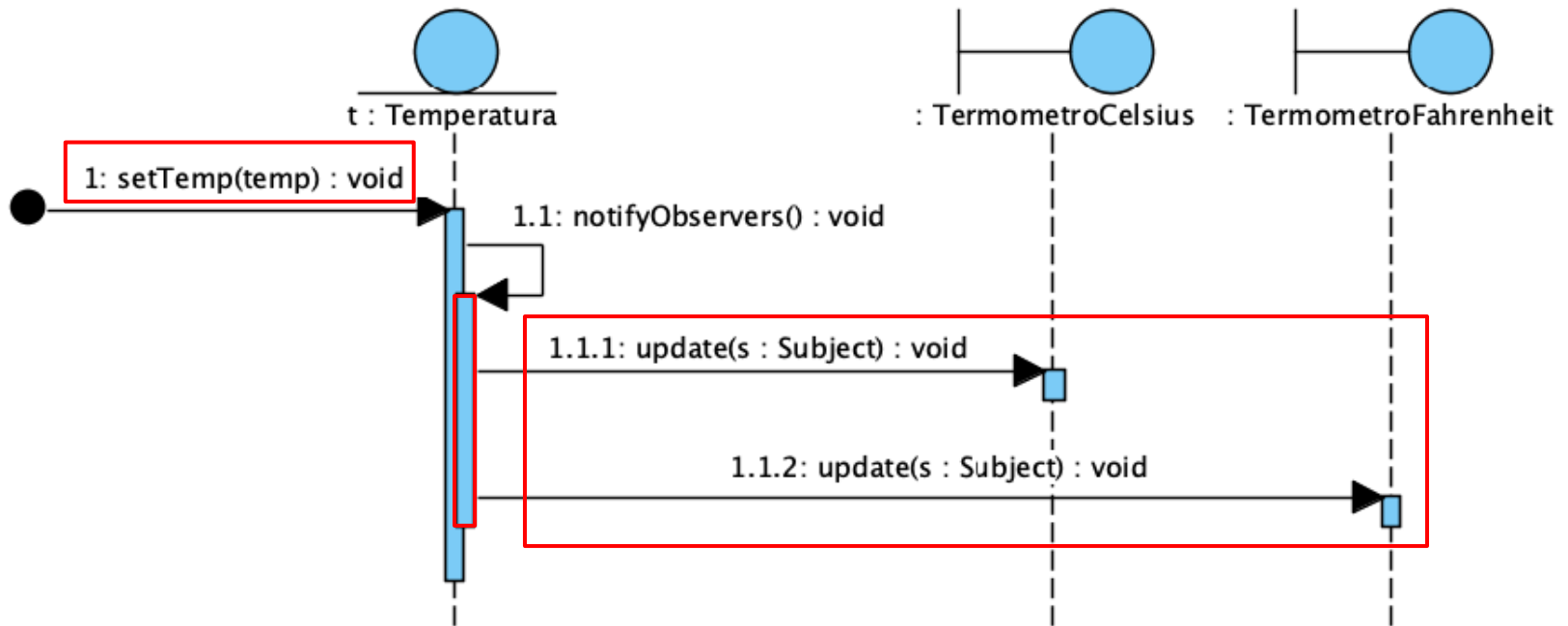
- Outra classe Termômetro

```
public class TermometroFahrenheit implements Observer {  
  
    public void update(Subject s) {  
        double temp = ((Temperatura) s).getTemp();  
        temp = temp * 1.8 + 32.0; //converte graus C para F  
        System.out.println("Temperatura Fahrenheit: " + temp);  
    }  
}
```

Exemplo UML



Exemplo UML



Demonstrar no Eclipse IDE
[PadraoDeProjetoObserver]

Código Observer:

<https://gist.github.com/mtov/5fadb0e599cb84fd6bd124b2ff37c03c>

PADRÕES DE PROJETO

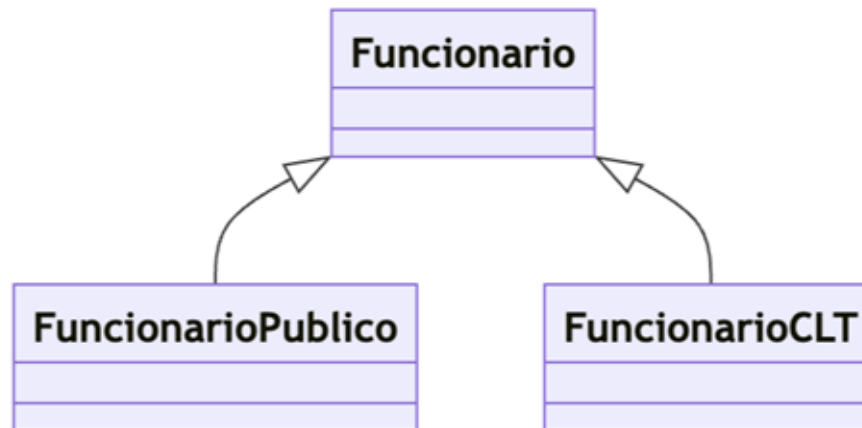
9. TEMPLATE METHOD

Padrão de projeto comportamental:
Flexibiliza interação e divisão de
responsabilidades entre classe e objetos



Contexto: Folha de Pagamento

- Uma classe base: `Funcionario`
- Duas subclasses: `FuncionarioPublico` e `FuncionarioCLT`





Problema

- Função que **calcula salário** de funcionários:
 - Passos são semelhantes para funcionários públicos e CLT
 - Porém, existem alguns detalhes diferentes
- Na **classe pai** (`Funcionario`) queremos definir o "**workflow principal**" (ou o *template*) para cálculo de salários – mínimo que seria seguido por todas as subclasses
- E deixar **aberto para as subclasses os refinamentos** desses passos



Solução: Padrão Template Method

- Especifica **como implementar o “esqueleto” de um algoritmo** em uma classe abstrata X
- Mas deixando pendente alguns passos (ou métodos abstratos) para serem implementados nas subclasses de X
- Ou seja, esse **padrão permite que subclasses customizem um algoritmo, mas sem mudar sua estrutura geral**, implementada na classe base



Solução: Padrão Template Method

```
abstract class Funcionario {
```

```
    double salario;
```

```
    ...
```

```
private abstract double calcDescontosPrevidencia();
```

```
private abstract double calcDescontosPlanoSaude();
```

```
private abstract double calcOutrosDescontos();
```

```
public double calcSalarioLiquido { // template method
```

```
    double prev = calcDescontosPrevidencia();
```

```
    double saude = calcDescontosPlanoSaude();
```

```
    double outros = calcOutrosDescontos();
```

```
    return salario - prev - saude - outros;
```

```
}
```

```
}
```

Serão
implementados pelas
subclasses

Padroniza a forma
de calcular salário
líquido

Passos principais (ou template, ou modelo) para cálculo de salário líquido. Porém Funcionário não sabe calcular os descontos, pois eles variam conforme o tipo de funcionário (público ou CLT).

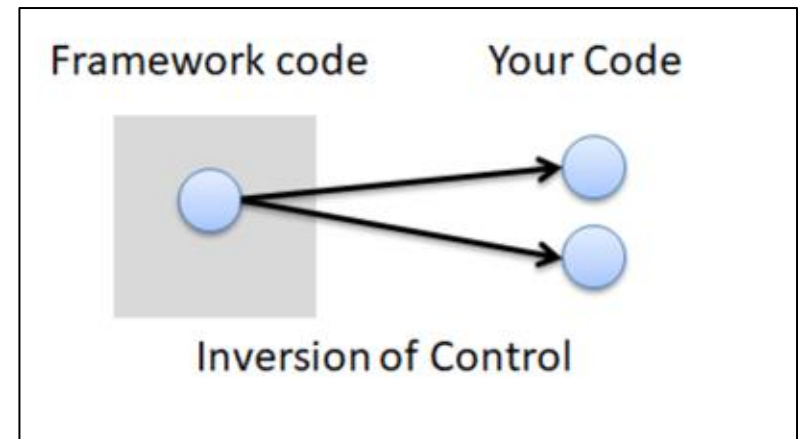
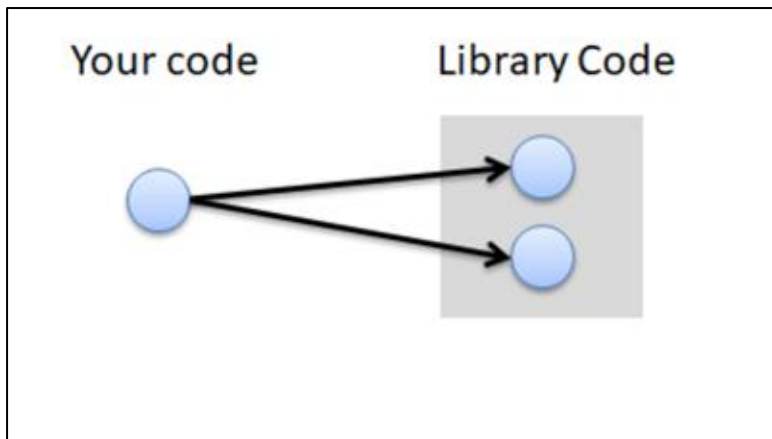


Solução: Padrão Template Method

- **Inversão de Controle**
 - *Template Method* é usado para implementar **Inversão de Controle**, principalmente em *frameworks*
 - Framework: define o "modelo" de um algoritmo/sistema
 - Mas clientes podem parametrizar alguns passos
 - Framework chama esse código dos clientes
 - Daí o termo **inversão de controle** (“código antigo chamando código novo”)



Frameworks vs Bibliotecas



Fonte: <https://github.com/prmr/SoftwareDesign/blob/master/modules/Module-06.md>

PADRÕES DE PROJETO

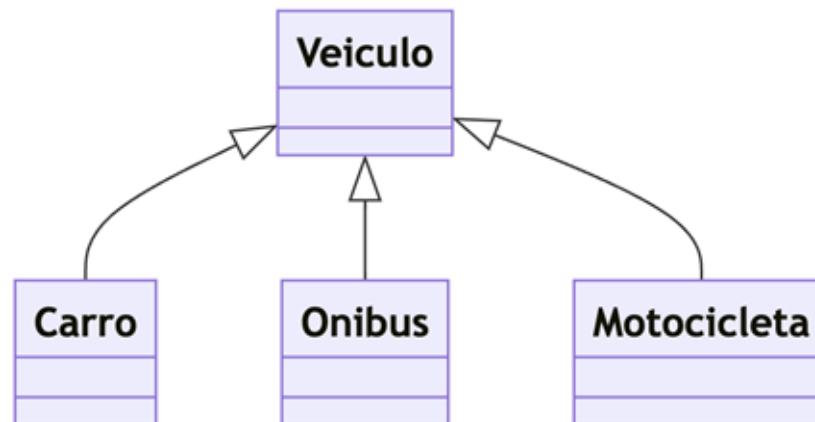
10. VISITOR

Padrão de projeto comportamental:
Flexibiliza interação e divisão de
responsabilidades entre classe e objetos



Contexto: Veículo e subclasses

- Suponha um sistema de estacionamentos com uma classe `Veiculo`
- E três subclasses: `Carro`, `Onibus` e `Bicicleta`





Contexto: Veiculo e subclasses

- E uma lista polimórfica de objetos Veiculo:

```
List<Veiculo> list = new ArrayList<Veiculo>();  
list.add(new Carro(..));  
list.add(new Onibus(..));  
...
```

Por que essa lista é chamada de polimórfica?



Problema

- Frequentemente, temos que realizar **uma operação com todos os veículos estacionados** (armazenados na lista polimórfica do slide anterior)
- Exemplo:
 - Imprimir dados de veículos (sejam `Carro`, `Onibus`,...)
 - Salvar dados de veículos em disco
 - Enviar uma mensagem para os donos dos veículos
 - Etc.



Problema

- Objetivo é **implementar essas operações fora das classes de Veículo**
- Queremos seguir o **princípio Aberto/Fechado**:
 - Manter `Veículo` e suas subclasses **fechados para mudanças**, mas **aberto para extensões**
 - **Extensões**: diversas operações que queremos realizar com veículos



Possível Alternativa

```
interface Visitor {  
    void visit(Carro c);  
    void visit(Onibus o);  
    void visit(Motocicleta m);  
}  
  
class PrintVisitor implements Visitor {  
    public void visit(Carro c) { "imprime dados de carro" }  
    public void visit(Onibus o) { "imprime dados de onibus" }  
    public void visit(Motocicleta m) {"imprime dados de moto"}  
}
```

PrintVisitor inclui métodos que são extensões para os Veículos!

Interface foi criada porque amanhã poderemos ter outros tipos de Visitors (exemplo: que salva os dados em um arquivo JSON)



Possível Alternativa (cont.)

Tentativa de visitar todos os veículos de uma lista polimórfica!

```
PrintVisitor visitor = new PrintVisitor();  
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {  
    visitor.visit(veiculo); // erro de compilação  
}
```

- Em Java, C++, C# e linguagens similares, compilador não conhece o tipo dinâmico do parâmetro `veiculo`, apenas o tipo estático (`Veiculo`)
- Objeto genérico está sendo passado como parâmetro onde era esperado um específico.
- Logo, ele não sabe "identificar" qual dos três métodos `visit` de `PrintVisitor` deverá ser chamado

Decisão do método a ser chamado é "bi-dimensional"

	PrintVisitor	MailVisitor	SaveVisitor	etc
Carro				
Onibus				
Motocicleta				
etc				



Solução: Padrão Visitor

- Permite **adicionar uma operação genérica em uma família de classes**, sem mexer no código delas.
 - Princípio Aberto/Fechado
- Permite simular *multiple dispatching*, em linguagens que oferecem apenas *single dispatching*
 - Despacho múltiplo considera simultaneamente o **tipo dinâmico do objeto polimórfico** (Ex: veículo) e **assinaturas dos métodos sobrecarregados** para escolher quem invocar.
 - Usado em linguagens antigas. Ex: Common Lisp



Solução: Padrão Visitor

```
abstract class Veiculo {  
    abstract public void accept(Visitor v);  
}  
  
class Carro extends Veiculo {  
    ...  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
    ...  
}  
  
class Onibus extends Veiculo {  
    ...  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
    ...  
}  
  
// Idem para Motocicleta
```

Método `accept` deverá ser implementado nas subclasses.

Chama método `visit`, do `Visitor`, mas agora passando um objeto estático como parâmetro.

Ex: `this` é `Carro`
 `this` é `Onibus`
 `this` é `Moto`



Solução: Padrão Visitor

```
PrintVisitor visitor = new PrintVisitor();  
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {  
    veiculo.accept(visitor);    //modificação do local onde havia erro de compilação  
                                //visitor.visit(veiculo);  
}
```

Chama accept do tipo dinâmico de veiculo. Suponha que seja Carro

```
class Carro extends Veiculo {  
    ...  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
    ...  
}
```

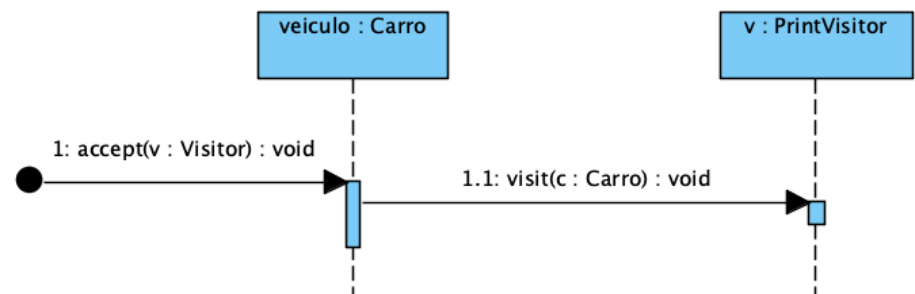
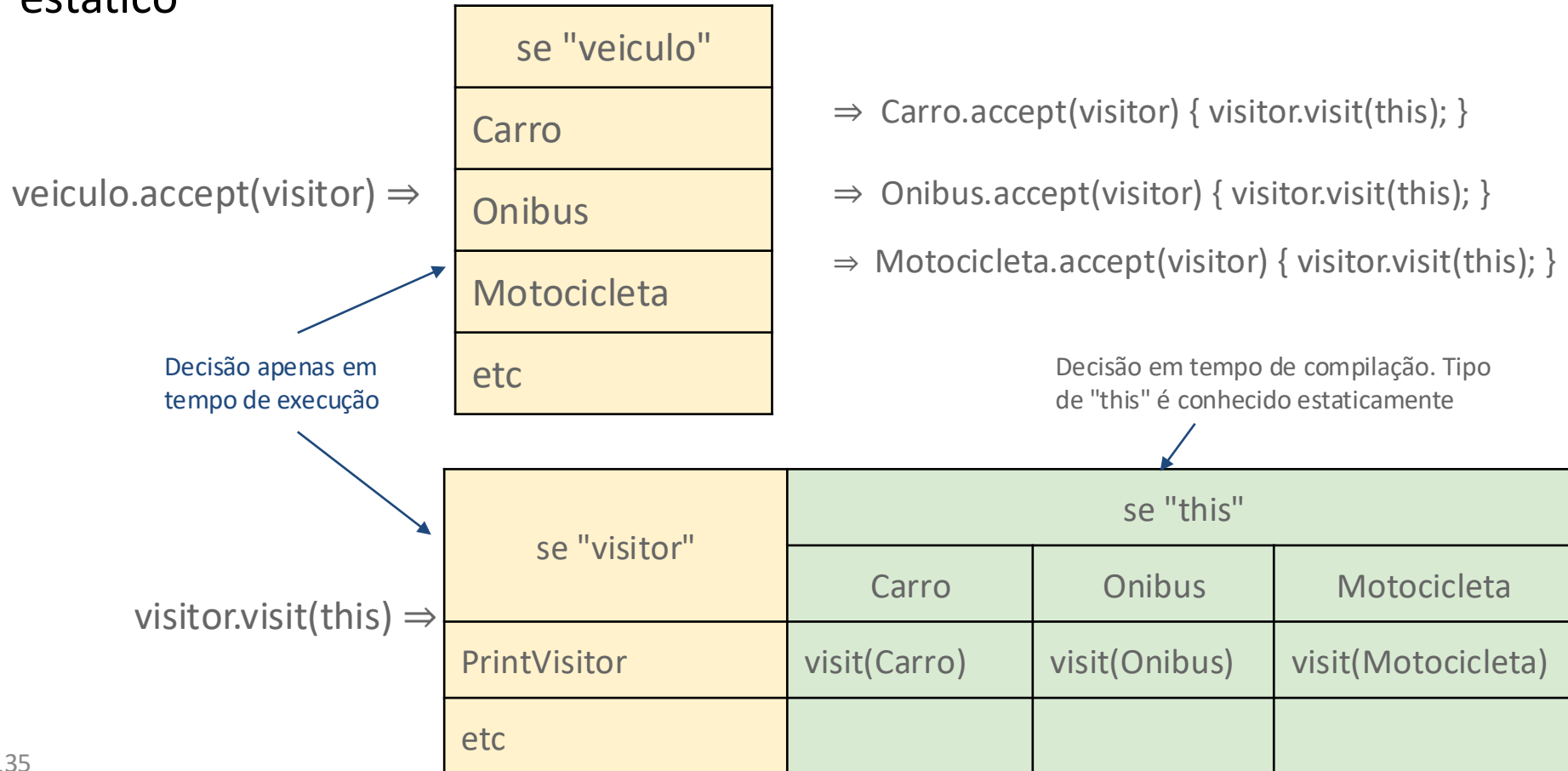


Diagrama de sequência

Tipo de `this` é conhecido estaticamente (Carro)

A escolha do método ocorre em dois passos dinâmicos, mais um passo estático



Visitor: vantagens & desvantagens



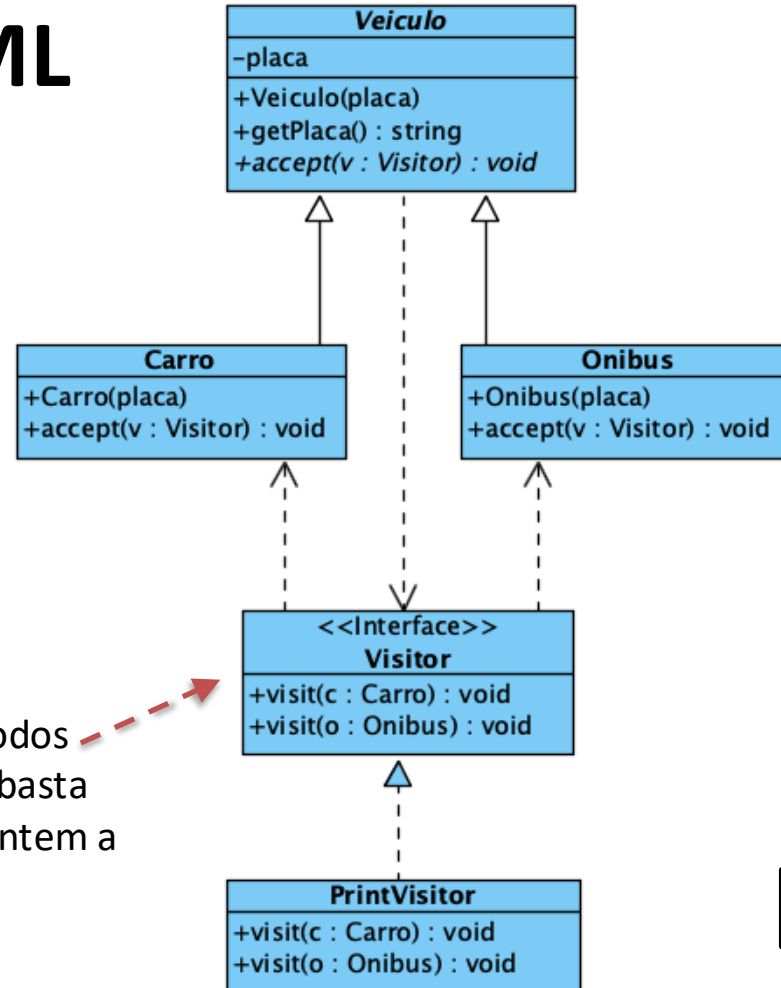
Padrão Visitor: Vantagens

- Visitors facilitam a adição de um método em uma hierarquia de classes
- Além da impressão de dados de `Veiculo` e de suas subclasses, poderiam existir outros Visitors, com outras operações:
 - Exemplo: calcular imposto de veículos de MG
- Muito prático para atender ao princípio Aberto/Fechado, criando extensões para as classes da hierarquia.



Padrão Visitor: Vantagens

Exemplo UML



Para criar novas extensões/métodos para uma hierarquia de classes, basta criar novas classes que implementem a interface `Visitor`. **Ex:** `CalcImpostoVisitor`

Estrutura do padrão



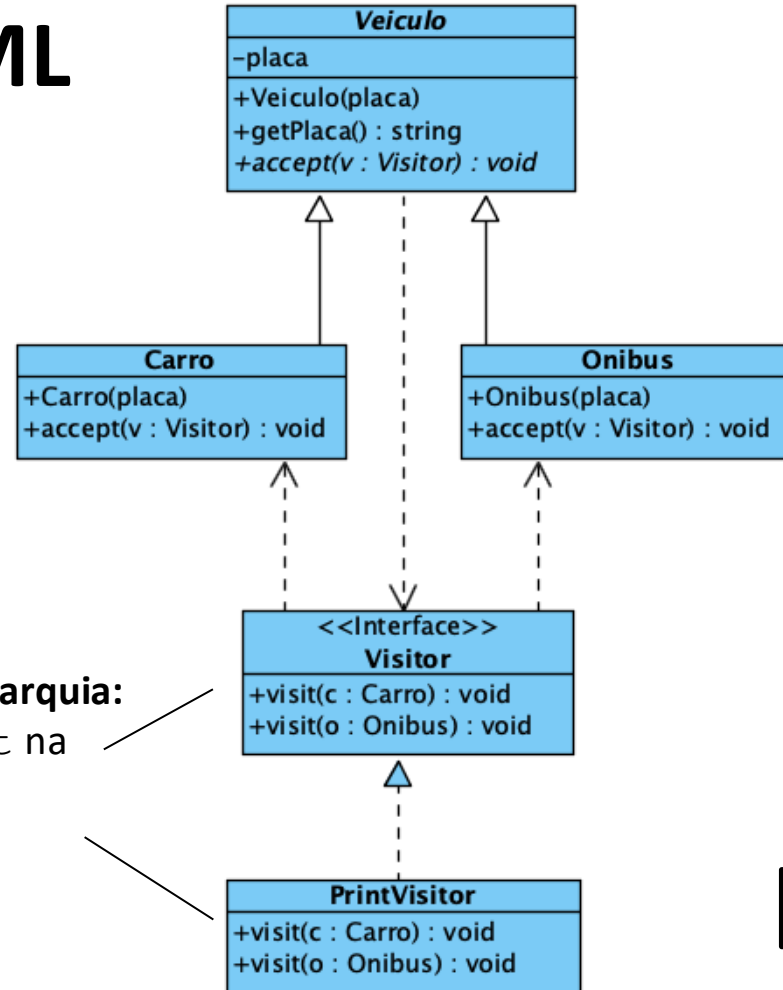
Padrão Visitor: Desvantagens

- A adição de uma nova classe na hierarquia (Ex: `Caminhao`), implica que a interface `Visitor` teria que ser atualizada, com a assinatura de um novo método `visit`
- Além disso, todos os `Visitors` (concretos) terão que ser atualizados, com a implementação de um novo método: `visit (Caminhao)`



Padrão Visitor: Desvantagens

Exemplo UML



Ao adicionar nova classe na hierarquia:

- Adicionar novo método `visit` na interface `Visitor`
- Atualizar todos os `Visitors` (concretos) existentes

Estrutura do padrão



Padrão Visitor: Desvantagens

- Visitors podem **quebrar encapsulamento**
 - Por exemplo, `Veiculo` pode ter que implementar **métodos públicos** expondo seu estado interno para que os Visitors tenham acesso a ele
 - Como uma classe não terá autonomia para executar as suas extensões, que serão executadas apenas ao receber uma visita, os visitantes precisarão ter visibilidade para estados que poderiam estar encapsulados nas classes.

Código Visitor:

<https://gist.github.com/mtov/d447fd22974c57688d0fd109cb9509ea>

**Quando não vale a pena usar
padrões de projeto?**



Paternite

- Termo usado por John Ousterhout: **inflamação causada pelo uso precipitado/exagerado de padrões de projeto – RISCO!!!**
- Padrões de Projeto $\Leftrightarrow$ *Design for change*
- **Design for change:** viabiliza projeto que acomoda mudanças futuras -> Tornar projetos **flexíveis**
- Se a chance de tais mudanças é zero, não precisamos de padrões de projeto
 - Não tente forçar um problema a caber em um padrão
 - Nem sempre ao usarmos padrões, estaremos melhorando um projeto



Trade-off: Design for Change vs Complexidade

- Não existe almoço grátis! Tudo tem um um preço...
- Para viabilizar "*design for change*", **Design Patterns** "**complicam**" um pouco o projeto
- Por exemplo, demandam a criação de classes extras, novas interfaces...

Exemplo: Strategy

Antes de incluir este padrão, devemos nos perguntar:

- Vamos precisar de parametrizar os algoritmos usados na implementação?
- Existem usuários que realmente vão precisar de algoritmos alternativos?

Solução sem padrão de projeto

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    public void sort() {  
        ... // ordena a lista usando Quicksort  
    }  
}
```

1 classe

Solução usando Strategy

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
}
```

1 classe (com mais código)
+1 classe abstrata
+2 classes de estratégias

```
abstract class SortStrategy {  
    abstract void sort(MyList list);  
}  
  
class QuickSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}  
  
class ShellSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}
```

Paternite: um (provável) exemplo

org.springframework.aop.framework

Class **AbstractSingletonProxyFactoryBean**

[java.lang.Object](#)

└ [org.springframework.aop.framework.ProxyConfig](#)

└ [org.springframework.aop.framework.AbstractSingletonProxyFactoryBean](#)

All Implemented Interfaces:

[Serializable](#), [BeanClassLoaderAware](#), [FactoryBean](#), [InitializingBean](#)

Direct Known Subclasses:

[TransactionProxyFactoryBean](#)

```
public abstract class AbstractSingletonProxyFactoryBean
```

```
extends ProxyConfig
```

```
implements FactoryBean, BeanClassLoaderAware, InitializingBean
```

Convenient proxy factory bean superclass for proxy factory beans that create only singletons.

Manages pre- and post-interceptors (references, rather than interceptor names, as in [ProxyFactoryBean](#)) and provides consistent interface management.

Atividades

Entrega: 25/05/2026



Exercícios – individual

Entrega: 25/05/2026

1. Liste os padrões de projeto que:
 - a) Ajudam a tornar uma classe aberta a extensões, sem que seja preciso modificar o seu código fonte (isto é, padrões que colocam em prática o princípio Aberto/Fechado).
 - b) Ajudam a desacoplar dois tipos de classes.
 - c) Ajudam a incrementar a coesão de uma classe (isto é, fazem com que a classe tenha Responsabilidade Única).
 - d) Simplificam o uso de um sistema.



Exercícios – individual

Entrega: 25/05/2026

2. Singleton é um dos padrões de projeto mais criticados e polêmicos.

Erich Gamma, um dos autores do livro GoF, já chegou inclusive a recomendar, em uma [entrevista](#), que ele deveria ser eliminado do catálogo:

> Erich: When discussing which patterns to drop, we found that we still love them all. (Not really—I'm in favor of dropping Singleton...)

Descreva porque esse padrão pode causar problemas de design, caso não seja usado de forma adequada.



Exercícios – individual

Entrega: 25/05/2026

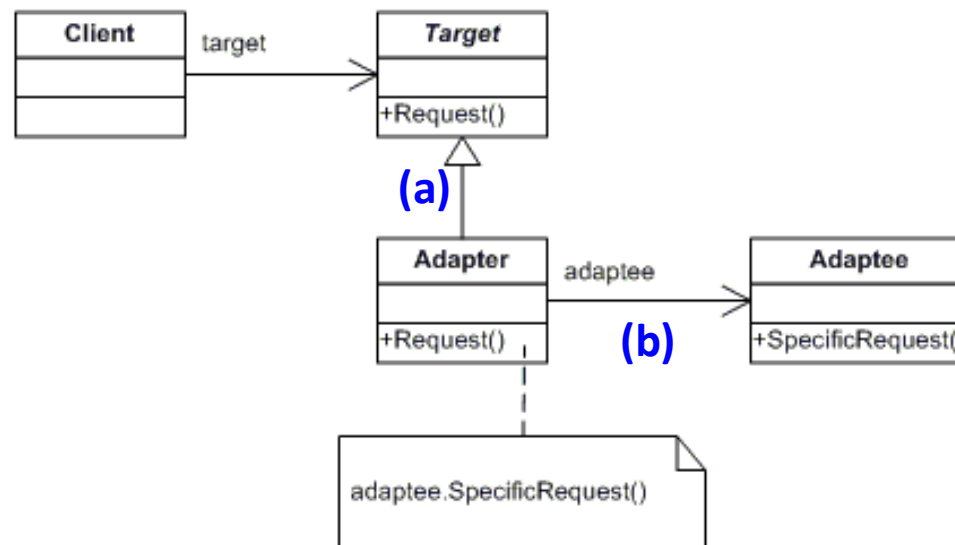
3. Além de otimização de desempenho via um cache, tal como visto nos slides, descreva outros três requisitos não-funcionais que podem ser implementados em um Proxy.



Exercícios – individual

Entrega: 25/05/2026

4. O seguinte diagrama de classes explica o padrão Adaptador.
- Em UML, qual o tipo da relação (a)? E qual o tipo da relação (b)?
 - Mapeie Target, Adapter e Adaptee para as interfaces e classes do exemplo de projetores visto em aula.





Exercícios – individual

Entrega: 25/05/2026

5. Cite três design patterns que ajudam nitidamente a tornar uma classe "aberta" para extensões? Isto é, favorecem um design que segue o Princípio Aberto/Fechado.

Trabalho – grupos de 3 ou 2 alunos



(9 grupos de 3 alunos e 1 grupo de 2 alunos)

Entrega: 25/05/2026

- Pesquisar uma aplicação prática (código) que utiliza o padrão de projeto e gravar um vídeo (aproximadamente 10 minutos) apresentando como ele é utilizado
 - Fábrica –
 - Singleton –
 - Proxy –
 - Adaptador –
 - Fachada –
 - Decorador -
 - Strategy –
 - Observador
 - Template Method –
 - Visitor –



Referência

- Valente, Marco Tulio. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade**. Leanpub, 2020. Capítulo 6. Disponível em:
<https://engsoftmoderna.info/cap6.html>